HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN I



BÁO CÁO BÀI TẬP LỚN
LẬP TRÌNH MẠNG

Đề tài:

"HỆ THỐNG HỌC NGỮ PHÁP TIẾNG ANH VÀ ĐỐI KHÁNG TRỰC TUYẾN GRAMMAR QUEST"

Giảng viên hướng dẫn: MSc. Ngô Văn Quyền

SINH VIÊN THỰC HIỆN

Lê Đình Thức	B23DCCN803
Đỗ Thị Thu Huế	B23DCCN355
Nguyễn Quang Minh	B23DCCN557
Nguyễn Ngọc Hoàng Nam	B23DCCN585
Trịnh Đức Mạnh	B23DCCN534

Hà Nội, tháng 11 năm 2026

NHẬN XÉT CỦA GIẢNG VIÊN HƯỚNG DẪN

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Điểm: ..... (*Bằng chữ:* .....)

Hà Nội, ngày ..... tháng ..... năm 20.....

Giảng viên hướng dẫn

MỤC LỤC

1 GIỚI THIỆU **1**

1.1 Bối cảnh và đặc trưng bài toán 1

1.2 Vấn đề nghiên cứu và các thuộc tính cần bảo toàn 1

1.3 Mục tiêu và câu hỏi nghiên cứu 2

1.4 Phương pháp nghiên cứu và phạm vi kiểm chứng 3

1.5 Phạm vi và đóng góp 4

2 CÁC CÔNG TRÌNH VÀ HỆ THỐNG LIÊN QUAN **6**

2.1 Phạm vi và tiêu chí khảo sát 6

2.2 Duolingo: tiếp cận học ngoại ngữ thông qua thực hành 6

2.3 Kahoot!: tiếp cận học tập thông qua hoạt động tương tác 7

2.4 Quizlet: tiếp cận học tập dựa trên học liệu ôn tập 8

2.5 British Council LearnEnglish: tiếp cận dựa trên nội dung ngữ pháp theo trình độ 8

2.6 Tổng hợp hạn chế và xác định khoảng trống 9

2.7 Kết luận 10

3 PHÁT BIỂU BÀI TOÁN **11**

3.1 Yêu cầu và phạm vi phục vụ 11

3.2 Mô hình trạng thái, thành phần và định danh 12

3.3 Mô hình lỗi và điều kiện khôi phục 13

3.4 Các bất biến và tiêu chí kiểm chứng 14

4 THIẾT KẾ VÀ TRIỂN KHAI HỆ THỐNG **15**

4.1 Thiết kế tổng thể hệ thống 15

4.2 Thiết kế cơ sở dữ liệu 17

4.3 Thiết kế và hiện thực các nghiệp vụ chính 19

4.4 Hiện thực giao diện và tích hợp hệ thống 23

4.5 Triển khai ứng dụng 24

4.6 Nhận xét về thiết kế và triển khai 26

5 HIỆN THỰC VÀ ĐỘ TIN CẬY 27

5.1 Ownership và chính sách thực thi theo kênh phiên 27
5.2 Ngân sách lập lịch và truyền trạng thái 29
5.3 Giao dịch và tính nhất quán của persistence 30
5.4 Vòng đời và xử lý tình huống cạnh tranh 31
5.5 Bảo mật và mô hình triển khai 33

6 THỰC NGHIỆM VÀ THẢO LUẬN 34

6.1 Thiết kế thực nghiệm và metric 34
6.2 Kết quả đối chứng đồng bộ và gom nhóm cập nhật 35
6.3 Kiểm chứng chức năng và tính nhất quán 37
6.4 Tải tham chiếu, kiểm soát sức chứa và phạm vi suy luận 38
6.5 Thảo luận kết quả và ưu tiên cải tiến 39

7 ĐÓNG GÓP VÀ KẾT LUẬN 42

7.1 Đóng góp và bằng chứng 42
7.2 Trả lời câu hỏi nghiên cứu và ý nghĩa kết quả 43
7.3 Giới hạn của nghiên cứu 44
7.4 Hướng phát triển 45
7.5 Kết luận 45

TÀI LIỆU THAM KHẢO 47

DANH MỤC HÌNH ẢNH

4.1 Kiến trúc client–server của hệ thống Grammar Quest 16
4.2 Quan hệ dữ liệu học tập và tiến độ (mô hình rút gọn) 18
4.3 Vòng đời biên soạn, xét duyệt và lưu phiên bản học liệu 21
4.4 Quy trình xử lý câu trả lời trong chế độ PvP 22
4.5 Sơ đồ kết nối các thành phần trong môi trường phát triển 25

5.1 Luồng bảo toàn tính nhất quán giao dịch kết hợp khóa bi quan, kiểm tra phiên bản lạc quan và đồng bộ thông báo `AFTER_COMMIT`. 30
5.2 Máy trạng thái vòng đời của Phòng đấu PvP (`arena_rooms`) và Quy trình xuất bản học liệu (`study_module_drafts`). 32

6.1 Đối chứng số khung tin WebSocket đẩy xuống mỗi kết nối trong 60 chu kỳ (`1.000 ms`/chu kỳ) giữa mô hình cơ sở và cơ chế `Diff-before-Send`. . . . 36
6.2 Cấu trúc phân tầng của hệ thống kiểm chứng tự động (E_V và E_C) trong dự án Grammar Quest. 38

DANH MỤC BẢNG BIỂU

3.1 Đối tượng sử dụng và chức năng chính. 11
3.2 Các tình huống lỗi và cách xử lý chính. 13
3.3 Bất biến cốt lõi và tình huống kiểm chứng. 14

5.1 Các tình huống giao thoa đồng thời tiêu biểu và cơ chế phân xử trong Grammar Quest. 28
5.2 Chính sách xử lý và tối ưu hóa theo từng loại dữ liệu bất đồng bộ trong Grammar Quest. 30

6.1 Định nghĩa hệ thống chỉ số thực nghiệm, phạm vi phản ánh hợp lệ và các diễn giải cần tránh. 35
6.2 Kết quả đối chứng định lượng giữa mô hình truyền trực tiếp và cơ chế gom nhóm / đệm ảnh chụp của Grammar Quest. 36
6.3 Kết quả thực nghiệm bộ ghép trận mở rộng biên độ cấp độ `ArenaMatchmaker` và bộ lọc đáp án `AssignmentAnswerGuard`. 37
6.4 Tổng hợp kết quả thực nghiệm tương tranh đa luồng, kiểm soát sức chứa và giới hạn tốc độ (E_C). 39
6.5 Ma trận tổng hợp kết quả kiểm chứng toàn hệ thống Grammar Quest theo phân hệ và câu hỏi nghiên cứu. 40

7.1 Cơ chế và căn cứ đối chiếu chính. 42

Chương 1

GIỚI THIỆU

1.1 Bối cảnh và đặc trưng bài toán

Việc tích hợp yếu tố trò chơi vào phần mềm giáo dục đặt ra yêu cầu kết hợp giữa quản lý hoạt động học tập và tổ chức tương tác giữa người học. Trong hệ thống học tập thông thường, phần lớn thao tác có thể được xử lý độc lập, chẳng hạn truy cập bài học, trả lời bài luyện tập hoặc xem tiến độ. Khi bổ sung thi đấu trực tuyến, thao tác của một người dùng có thể làm thay đổi trạng thái mà người dùng khác đang quan sát. Bài toán vì vậy mở rộng từ lưu trữ kết quả cá nhân sang phối hợp nhiều chủ thể trong cùng một tiến trình tương tác.

Grammar Quest được xây dựng trong bối cảnh này. Dự án là nền tảng học ngữ pháp tiếng Anh bằng tiếng Việt, cung cấp lộ trình học, luyện tập, kiểm tra đầu vào, nhiệm vụ hằng ngày, quản lý lớp học và đối kháng PvP. Hệ thống phục vụ ba nhóm người dùng gồm học sinh, giáo viên và quản trị viên. Sự phân chia này hình thành các yêu cầu khác nhau về quyền truy cập, trách nhiệm tạo nội dung và phạm vi thao tác đối với dữ liệu.

Đặc trưng đáng chú ý của dự án là sự tồn tại đồng thời của nghiệp vụ học tập có tính tích lũy và nghiệp vụ thi đấu có tính thời điểm. Tiến độ học tập cần được lưu trữ bền vững trong thời gian dài, trong khi trạng thái trận đấu phải được xử lý theo thứ tự và thời hạn cụ thể. Một câu trả lời có thể hợp lệ về nội dung nhưng không còn hợp lệ nếu được gửi đến sau khi vòng đấu kết thúc. Tương tự, một thao tác đúng quyền vẫn cần bị từ chối khi trạng thái phòng không còn cho phép thực hiện.

Về công nghệ, dự án sử dụng React và TypeScript cho giao diện, Java 21 và Spring Boot cho máy chủ, MySQL cho lưu trữ dữ liệu và Flyway cho quản lý thay đổi cấu trúc cơ sở dữ liệu. REST API và WebSocket cùng tham gia trao đổi dữ liệu. Kiến trúc này tạo điều kiện kết hợp các thao tác theo yêu cầu với cập nhật chủ động từ máy chủ, nhưng cũng đặt ra vấn đề bảo đảm các đường tương tác tuân thủ cùng một hệ quy tắc nghiệp vụ.

1.2 Vấn đề nghiên cứu và các thuộc tính cần bảo toàn

Vấn đề nghiên cứu trung tâm là cách tổ chức thực thi và đồng bộ trạng thái trong một ứng dụng học tập có đối kháng trực tuyến, sao cho kết quả nghiệp vụ vẫn đúng khi xuất hiện yêu cầu đồng thời, dữ liệu đầu vào không hợp lệ hoặc gián đoạn kết nối. Đối với phân hệ PvP, tính đúng đắn không chỉ phụ thuộc vào phép tính điểm mà còn phụ thuộc vào danh tính người gửi, trạng thái phòng, vòng đấu hiện tại và thời điểm máy chủ tiếp nhận thao tác.

Thuộc tính đầu tiên cần bảo toàn là tính toàn vẹn của kết quả. Mỗi câu trả lời chỉ được ghi nhận theo quy tắc của vòng tương ứng; việc gửi lại yêu cầu không được tạo ra tác động tích lũy ngoài ý muốn. Mã nguồn thể hiện định hướng này thông qua xử lý câu trả lời đã tồn tại: gửi lại cùng lựa chọn có thể trả về trạng thái hiện tại, trong khi thay đổi lựa chọn sau khi đã trả lời bị từ chối. Cơ chế đó hạn chế tác động của thao tác lặp, nhưng chưa thể được khái quát thành bảo đảm xử lý đúng một lần cho mọi loại yêu cầu trong hệ thống.

Thuộc tính thứ hai là tính nhất quán giữa các bên tham gia. Do độ trễ truyền thông, hai trình duyệt không nhất thiết hiển thị cùng một trạng thái tại mọi thời điểm. Yêu cầu phù hợp hơn là các bên phải hội tụ về cùng trạng thái chính thức sau khi nhận và xử lý cập nhật. Việc sử dụng `stateVersion` trong dữ liệu phòng đấu cho thấy hệ thống có phân biệt thứ tự trạng thái, qua đó hỗ trợ hạn chế cập nhật cũ ghi đè thông tin mới.

Thuộc tính thứ ba liên quan đến thời gian và quyền quyết định. Trình duyệt cần hiển thị đồng hồ để người chơi theo dõi, nhưng thời điểm hết hạn phải được xác định tại máy chủ. Nếu quyền quyết định được đặt ở máy khách, việc thay đổi đồng hồ hoặc can thiệp mã giao diện có thể ảnh hưởng đến luật thi đấu. Do đó, thời gian hiển thị và thời gian dùng để xét tính hợp lệ cần được xem là hai thành phần có trách nhiệm khác nhau.

Bên cạnh đó, hệ thống phải bảo toàn ranh giới truy cập và tính bảo mật của dữ liệu câu hỏi. Người dùng không thuộc phòng không được tác động đến trận đấu; dữ liệu gửi trước thời điểm công bố kết quả không được làm lộ đáp án đúng. Những thuộc tính này gắn trực tiếp với độ tin cậy của hoạt động đối kháng, bởi một kết quả được tính chính xác vẫn không có ý nghĩa nếu người chơi có thể truy cập thông tin ngoài phạm vi cho phép.

1.3 Mục tiêu và câu hỏi nghiên cứu

Mục tiêu tổng quát của đề tài là xây dựng hệ thống Grammar Quest, tích hợp học ngữ pháp tiếng Anh theo lộ trình, quản lý lớp học và thi đấu đối kháng trực tuyến. Hệ thống cần phối hợp các chức năng này trên kiến trúc client–server, đồng thời kiểm soát quyền truy cập, bảo vệ kết quả học tập và duy trì trạng thái nhất quán khi nhiều người dùng tương tác qua mạng.

Về mặt kỹ thuật, đề tài tập trung vào bốn mục tiêu: giảm xử lý và truyền dữ liệu dư thừa trong quá trình đồng bộ; bảo toàn các bất biến nghiệp vụ khi tiếp nhận yêu cầu không hợp lệ hoặc gửi lặp; xử lý những thao tác cập nhật đồng thời; và kiểm soát tài nguyên khi số kết nối hoặc tần suất yêu cầu vượt giới hạn. Từ đó, báo cáo xác định bốn câu hỏi nghiên cứu như sau:

RQ1 — Hiệu quả đồng bộ và giảm tải dư thừa: Cơ chế gom nhóm DATA_INVALIDATED, bộ đệm SnapshotKey và kho gói câu hỏi ArenaSnapshotStore giúp giảm số lần mã hóa JSON, số thông điệp WebSocket và số truy vấn nạp dữ liệu như thế nào so với mô hình xử lý trực tiếp theo từng sự kiện hoặc từng kết nối?

Câu hỏi này tập trung đánh giá mức độ hạn chế công việc lặp lại khi nhiều tab theo dõi cùng tài nguyên, nhiều thay đổi phát sinh gần nhau hoặc trạng thái không thay đổi qua các chu kỳ cập nhật.

RQ2 — Tính đúng đắn giao dịch và bất biến nghiệp vụ: Các cơ chế khóa bi quan, kiểm tra phiên bản, khóa chống lặp và thông báo sau khi giao dịch hoàn tất có bảo toàn những bất biến nghiệp vụ đã xác định trước các thao tác không hợp lệ, dữ liệu cũ hoặc yêu cầu gửi trùng hay không?

Phạm vi xem xét gồm quyền truy cập, tính nhất quán giữa đề và bài làm, việc bảo vệ kết quả đã ghi nhận và ngăn tác dụng nghiệp vụ bị thực hiện lặp ngoài ý muốn. Mức độ bảo toàn được đánh giá trong các tình huống kiểm thử cụ thể.

RQ3 — Xử lý tương tranh và phân xử xung đột trực tiếp: Khi 20 luồng học viên cùng lưu hoặc nộp bài đồng thời với thao tác sửa đề của giáo viên, hệ thống có duy trì kết quả chấm nhất quán theo phiên bản đề được áp dụng hay không; có ghi nhận mất cập nhật hoặc deadlock trong quá trình kiểm thử hay không?

Câu hỏi này xem xét sự phối hợp giữa phạm vi giao dịch, khóa trên tài nguyên dùng chung và cơ chế chấm lại. Kết quả cần được đối chiếu với dữ liệu đã lưu để xác định tính đúng đắn của từng bài nộp.

RQ4 — Kiểm soát sức chứa và cô lập hành vi lạm dụng: Khi số kết nối vượt hạn mức hoặc một tài khoản gửi câu trả lời vượt giới hạn 10 lượt/giây, hệ thống có từ chối đúng yêu cầu vượt ngưỡng, duy trì hoạt động của tài khoản khác và thu hồi tài nguyên để tiếp nhận yêu cầu hợp lệ khi tải giảm hay không?

Câu hỏi này tập trung vào cơ chế ServerCapacity và AnswerRateLimiter trong phạm vi triển khai một máy chủ. Việc đánh giá phân biệt giới hạn cấu hình với năng lực phục vụ thực tế, không suy trực tiếp số kết nối cho phép thành số người dùng có thể hoạt động đồng thời.

Bốn câu hỏi trên là cơ sở lựa chọn kịch bản và chỉ số đánh giá tại chương 6. Các kết quả được tổng hợp tại chương 7, kèm theo điều kiện áp dụng và giới hạn của phạm vi kiểm chứng.

1.4 Phương pháp nghiên cứu và phạm vi kiểm chứng

Báo cáo kết hợp phân tích mã nguồn, mô hình hóa trạng thái và kiểm thử để đánh giá thiết kế của hệ thống Grammar Quest. Phạm vi nghiên cứu tập trung vào cơ chế giao tiếp thời gian thực, tính nhất quán của dữ liệu khi có thao tác đồng thời và việc kiểm soát tài nguyên trong môi trường triển khai một máy chủ.

Trước hết, phân tích tĩnh được sử dụng để xác định trách nhiệm của các thành phần, luồng xử lý yêu cầu và những cơ chế bảo vệ đã được triển khai. Các thành phần được xem xét gồm tầng tiếp nhận REST và WebSocket, xử lý nghiệp vụ, truy cập cơ sở dữ liệu, quản lý phiên và cập nhật giao diện. Việc đối chiếu mã nguồn với cấu trúc dữ liệu và các migration giúp làm rõ phạm vi giao dịch, thứ tự khóa, kiểm tra quyền, quản lý phiên bản và xử lý yêu cầu lặp.

Tiếp theo, các nghiệp vụ có trạng thái được phân tích thông qua điều kiện chuyển trạng thái và những bất biến cần bảo toàn. Đối với PvP, nội dung xem xét gồm quyền tham gia, vòng đấu hiện tại, thời hạn trả lời, điều kiện tạm dừng và kết thúc trận. Đối với kiểm tra trực tiếp và quản lý học liệu, trọng tâm là quan hệ giữa phiên bản nội dung, câu trả lời, kết quả chấm và lịch sử cập nhật. Các bất biến này được sử dụng làm căn cứ xác định kết quả mong đợi của từng kịch bản kiểm thử.

Quá trình kiểm chứng được tổ chức ở ba mức. Kiểm thử đơn vị đánh giá các quy tắc và thành phần riêng lẻ. Kiểm thử tích hợp với MySQL kiểm tra sự phối hợp giữa nghiệp vụ, giao dịch và dữ liệu đã lưu. Các kịch bản trình duyệt kiểm tra luồng tương tác, trao đổi thông điệp và cập nhật giao diện. Những tình huống được lựa chọn bao gồm thao tác hợp lệ, yêu cầu gửi lặp, dữ liệu không hợp lệ, cập nhật đồng thời và vượt giới hạn kết nối hoặc tốc độ gửi.

Đối với hiệu quả đồng bộ, báo cáo đối chiếu cơ chế gom nhóm và chia sẻ snapshot với mô hình cơ sở trong các kịch bản xác định. Các đại lượng quan sát gồm số lần truy vấn, số lần mã hóa JSON và số thông điệp được gửi. Đối với tính nhất quán và kiểm soát tài nguyên, kết quả được đánh giá thông qua trạng thái bản ghi sau xử lý, điểm số, phản hồi từ chối và số tài nguyên được cấp phát hoặc thu hồi. Môi trường, điều kiện thực hiện và kết quả của các kịch bản được trình bày tại chương 6.

Phạm vi kết luận được giới hạn bởi những kiểm thử đã thực thi và điều kiện của môi trường đánh giá. Kết quả đạt trong một tập kịch bản không chứng minh hệ thống đúng trong mọi tình huống hoặc không thể xảy ra deadlock. Tương tự, việc giảm số lần xử lý và thông điệp không tự xác định mức giảm CPU, RAM hay độ trễ toàn hệ thống. Báo cáo chưa đánh giá đầy đủ hiệu năng trên mạng diện rộng, khả năng vận hành nhiều backend hoặc hiệu quả giáo dục đối với người học; các nội dung này được xác định là giới hạn và hướng nghiên cứu tiếp theo.

1.5 Phạm vi và đóng góp

Phạm vi hệ thống bao gồm nhiều nghiệp vụ giáo dục, nhưng trọng tâm phân tích của báo cáo là cơ chế thực thi và đồng bộ trong chế độ PvP. Các chức năng học tập, lớp học và quản trị được xem là bối cảnh nghiệp vụ có liên quan đến tài khoản, phân quyền và dữ liệu dùng chung. Báo cáo không đánh giá tính đầy đủ của giáo trình hoặc hiệu quả sư phạm thông qua khảo sát người học.

Đóng góp của dự án trước hết nằm ở việc tích hợp các hoạt động học tập và đối kháng trong cùng một nền tảng. Ở mức kỹ thuật, dự án tổ chức xử lý trạng thái thi đấu tại máy chủ, kết hợp giao dịch cơ sở dữ liệu với cập nhật qua WebSocket, đồng thời sử dụng phiên bản trạng thái và dữ liệu câu hỏi theo phòng để hỗ trợ quá trình đồng bộ.

Các đóng góp này có tính chất thiết kế và triển khai hệ thống. Chưa có cơ sở để xem dự án là một thuật toán game mạng mới hoặc kết luận rằng yếu tố đối kháng đã cải thiện kết quả học tập. Giá trị nghiên cứu có thể được phát triển từ việc kiểm chứng có hệ

thống các cơ chế hiện có, xác định điều kiện chúng hoạt động đúng và làm rõ giới hạn của kiến trúc.

Chương 2

CÁC CÔNG TRÌNH VÀ HỆ THỐNG LIÊN QUAN

2.1 Phạm vi và tiêu chí khảo sát

Các nền tảng học tập trực tuyến đã phát triển theo nhiều hướng, từ hỗ trợ học ngoại ngữ cá nhân đến cung cấp công cụ ôn tập và tổ chức hoạt động tương tác trong lớp học. Đối với bài toán của Grammar Quest, việc khảo sát cần xem xét đồng thời hai khía cạnh: tổ chức quá trình học ngữ pháp và tạo môi trường tương tác giữa người học với hệ thống, giáo viên và những người học khác.

Bốn hệ thống được lựa chọn để phân tích gồm Duolingo, Kahoot!, Quizlet và British Council LearnEnglish. Duolingo đại diện cho nền tảng học ngoại ngữ tích hợp yếu tố trò chơi; Kahoot! đại diện cho công cụ tổ chức hoạt động trả lời câu hỏi tương tác; Quizlet tập trung vào học liệu và ôn tập theo nhiều hình thức; British Council LearnEnglish cung cấp tài nguyên học tiếng Anh, trong đó có nội dung ngữ pháp được phân chia theo trình độ.

Việc phân tích tập trung vào cách tổ chức kiến thức, hình thức luyện tập, khả năng hỗ trợ người học Việt Nam và mối liên hệ giữa học tập cá nhân với hoạt động lớp học. Các hạn chế được xem xét trong phạm vi đáp ứng bài toán nghiên cứu, không được hiểu là đánh giá chất lượng tổng thể của từng nền tảng. Bên cạnh đó, khảo sát dựa trên tài liệu công khai về chức năng và cách sử dụng, do đó không đưa ra kết luận về kiến trúc nội bộ hoặc hiệu năng vận hành của các hệ thống.

2.2 Duolingo: tiếp cận học ngoại ngữ thông qua thực hành

Duolingo tổ chức việc học ngôn ngữ thông qua các hoạt động thực hành và phản hồi. Đối với ngữ pháp, nền tảng kết hợp việc tiếp xúc với cấu trúc trong bài tập cùng các hình thức giải thích trực tiếp. Tài liệu chính thức đề cập đến hướng dẫn ngữ pháp và bài học tập trung vào cấu trúc; tính năng *Explain My Answer* cung cấp giải thích liên quan đến lỗi ngữ pháp và từ vựng. Vì vậy, việc đánh giá Duolingo là hệ thống chỉ yêu cầu ghi nhớ đáp án mà không giải thích quy tắc là thiếu chính xác.

Nguồn: Duolingo, *Does Duolingo Teach Grammar?*, *Explain My Answer*.

Ưu điểm của cách tiếp cận này là đặt kiến thức trong hoạt động sử dụng ngôn ngữ. Người học tiếp xúc với cấu trúc qua câu hỏi và nhận phản hồi trong quá trình thực hành. Điều đó tạo sự liên kết giữa kiến thức và tình huống vận dụng, thay vì chỉ trình bày quy tắc dưới dạng tài liệu để đọc.

Tuy nhiên, mục tiêu học ngoại ngữ tổng hợp không hoàn toàn trùng với nhu cầu học ngữ pháp theo chuyên đề. Trong bối cảnh lớp học, người học có thể cần tập trung vào một cấu trúc cụ thể, so sánh các cấu trúc dễ nhầm lẫn hoặc luyện tập theo nội dung giáo

viên vừa giảng dạy. Việc đáp ứng những nhu cầu này phụ thuộc vào mức độ tương ứng giữa cách sắp xếp nội dung của nền tảng và chương trình mà người học đang theo đuổi.

Hạn chế cần xem xét ở đây là khả năng điều chỉnh quá trình học theo yêu cầu chuyên biệt. Một lộ trình học ngoại ngữ được xây dựng sẵn có thể thuận tiện cho việc học theo chương trình của nền tảng, nhưng chưa thể mặc nhiên đáp ứng trình tự kiến thức của từng lớp học. Tương tự, giải thích cho một câu trả lời cụ thể có giá trị phản hồi tức thời, song chưa tự bảo đảm người học hình thành được cái nhìn hệ thống về quan hệ giữa các chủ điểm ngữ pháp.

Từ góc độ bài toán nghiên cứu, Duolingo cho thấy thực hành và phản hồi là những thành phần quan trọng của quá trình học. Vấn đề còn cần làm rõ là cách tổ chức chúng trong một chương trình ngữ pháp bằng tiếng Việt, có khả năng gắn với nội dung giảng dạy và nhu cầu quản lý của giáo viên.

2.3 Kahoot!: tiếp cận học tập thông qua hoạt động tương tác

Kahoot! cung cấp các hình thức tổ chức hoạt động học tập như phiên chơi trực tiếp, giao khóa học và tự học. Phạm vi sử dụng của nền tảng vì vậy rộng hơn việc tổ chức câu hỏi tại lớp dưới sự điều khiển trực tiếp của giáo viên.

Nguồn: Kahoot!, *Courses: How to assign, host, and study.*

Điểm nổi bật của cách tiếp cận này là biến hoạt động trả lời câu hỏi thành một quá trình có tương tác. Giáo viên có thể sử dụng câu hỏi để kiểm tra mức độ tiếp nhận kiến thức, còn người học tham gia trong một khuôn khổ hoạt động chung. Cách tổ chức đó phù hợp với nhu cầu kiểm tra nhanh và tạo thêm hình thức tham gia trong lớp học.

Tuy nhiên, Kahoot! là công cụ phục vụ nhiều lĩnh vực. Nền tảng cung cấp phương tiện tổ chức hoạt động, còn tính hệ thống của nội dung ngữ pháp phụ thuộc vào cách người biên soạn xây dựng câu hỏi và sắp xếp chúng. Việc có nhiều bài kiểm tra không đồng nghĩa với việc đã hình thành một chương trình học có quan hệ tiên quyết, sự tăng dần về độ khó và cơ chế củng cố kiến thức.

Giới hạn này trở nên rõ hơn khi phân biệt kết quả của một phiên hoạt động với năng lực dài hạn của người học. Trả lời đúng trong một phiên kiểm tra có thể phản ánh khả năng nhận diện kiến thức tại thời điểm đó, nhưng chưa đủ để kết luận người học có thể vận dụng cấu trúc trong ngữ cảnh mới. Nếu hoạt động không gắn với nội dung giải thích và ôn tập tiếp theo, dữ liệu trả lời chủ yếu phục vụ đánh giá phiên học thay vì mô tả đầy đủ quá trình hình thành kiến thức.

Ngoài ra, với các hoạt động có giới hạn thời gian, kết quả có thể chịu tác động của tốc độ đọc và phản ứng bên cạnh kiến thức. Đây là giới hạn phương pháp cần cân nhắc khi sử dụng trò chơi hỏi đáp để đánh giá ngữ pháp, không phải bằng chứng rằng mọi hoạt động trên Kahoot! đều đánh giá thiếu chính xác.

Như vậy, vấn đề đặt ra từ cách tiếp cận của Kahoot! là mối liên hệ giữa tương tác và quá trình học có tính kế thừa. Một hệ thống chuyên biệt cần làm rõ hoạt động trả lời

câu hỏi thuộc chủ điểm nào, kết quả phản ánh điều gì và được sử dụng ra sao trong quá trình học tiếp theo.

2.4 Quizlet: tiếp cận học tập dựa trên học liệu ôn tập

Quizlet cung cấp các chế độ như *Flashcards*, *Learn*, *Test*, *Match* và *Quizlet Live*. Cùng một bộ học liệu có thể được sử dụng để xem lại kiến thức, luyện tập, tự kiểm tra hoặc tham gia hoạt động cộng tác. *Learn* hỗ trợ luyện tập dựa trên mức độ quen thuộc với bộ học liệu, trong khi *Flashcards* tổ chức nội dung dưới dạng thuật ngữ–định nghĩa hoặc câu hỏi–đáp án.

Nguồn: Quizlet, *Study Modes*.

Ưu điểm của mô hình này là tính linh hoạt và khả năng tái sử dụng nội dung. Người học có thể tiếp cận cùng kiến thức qua nhiều hình thức mà không cần xây dựng lại học liệu cho từng hoạt động. Đối với những nội dung có thể biểu diễn thành các đơn vị tương đối độc lập, cấu trúc này tạo thuận lợi cho việc ôn tập.

Tuy nhiên, kiến thức ngữ pháp không chỉ là tập hợp các cặp thông tin. Việc hiểu một cấu trúc thường đòi hỏi xác định điều kiện sử dụng, ý nghĩa trong ngữ cảnh, ngoại lệ và sự khác biệt với những cấu trúc gần nhau. Khi học liệu được chia thành các đơn vị ngắn, quan hệ giữa chúng cần được người biên soạn thể hiện bổ sung. Nếu thiếu sự liên kết đó, người học có thể ghi nhớ từng đáp án nhưng chưa nắm được cơ sở lựa chọn khi gặp tình huống mới.

Một giới hạn khác nằm ở quan hệ giữa bộ học liệu và lộ trình kiến thức. Cơ chế điều chỉnh luyện tập theo mức độ quen thuộc với một bộ nội dung không đồng nhất với việc xác định người học đã có đủ kiến thức nền để chuyển sang chủ điểm tiếp theo. Hai vấn đề này cần được phân biệt khi xây dựng hệ thống ngữ pháp theo cấp độ.

Do đó, Quizlet đáp ứng rõ nhu cầu tổ chức và ôn tập học liệu, nhưng việc hình thành chương trình ngữ pháp có cấu trúc vẫn cần hoạt động thiết kế nội dung bổ sung. Khoảng trống liên quan đến đề tài là sự kết nối giữa lý thuyết, điều kiện tiên quyết, luyện tập và đánh giá trong cùng một tiến trình học.

2.5 British Council LearnEnglish: tiếp cận dựa trên nội dung ngữ pháp theo trình độ

British Council LearnEnglish cung cấp tài nguyên ngữ pháp gồm phần giải thích và bài tập tương tác. Nội dung được tổ chức theo các nhóm trình độ như A1–A2, B1–B2 và C1, đồng thời có khu vực tham khảo quy tắc ngữ pháp. Cách phân chia này tạo cơ sở để người học lựa chọn nội dung phù hợp với trình độ và nhu cầu.

Nguồn: British Council LearnEnglish, *Grammar*.

So với công cụ hỏi đáp đa mục đích, phần tài nguyên này thể hiện rõ sự tập trung vào nội dung ngôn ngữ. Việc kết hợp giải thích với bài tập giúp người học có thể tiếp cận

quy tắc rồi kiểm tra mức độ hiểu. Đây là điểm phù hợp với yêu cầu học ngữ pháp theo chủ đề.

Tuy nhiên, các trang tài nguyên được khảo sát trình bày nội dung bằng tiếng Anh. Với người Việt có nền tảng hạn chế, việc hiểu lời giải thích có thể đòi hỏi thêm khả năng đọc và vốn từ trước khi tiếp cận được quy tắc. Hạn chế này phụ thuộc trình độ người dùng: cùng một hình thức trình bày có thể phù hợp với người học trung cấp nhưng tạo thêm khó khăn cho người mới bắt đầu.

Ngoài ra, việc truy cập tài nguyên theo trình độ chưa tự xác lập mối liên hệ với chương trình của một lớp học cụ thể. Trong phạm vi phần ngữ pháp được khảo sát, trọng tâm là cung cấp nội dung và thực hành. Các yêu cầu như giao bài theo lớp, theo dõi hoạt động và liên kết kết quả với đối kháng trực tuyến cần được xem xét riêng. Nhận định này chỉ áp dụng cho phạm vi tài nguyên đang phân tích, không suy rộng thành kết luận về toàn bộ dịch vụ của British Council.

Hệ thống này vì vậy cho thấy một nền tảng có nội dung phân chia rõ ràng vẫn có thể còn khoảng cách về ngôn ngữ hỗ trợ và bối cảnh sử dụng đối với nhóm người học cụ thể. Đối với đề tài, vấn đề cần giải quyết là kết hợp tính hệ thống của nội dung với khả năng tiếp cận của người Việt và các hoạt động quản lý học tập.

2.6 Tổng hợp hạn chế và xác định khoảng trống

Các hệ thống được khảo sát tiếp cận bài toán từ những đơn vị tổ chức khác nhau. Duolingo đặt trọng tâm vào quá trình học ngoại ngữ; Kahoot! tổ chức hoạt động tương tác; Quizlet khai thác bộ học liệu; British Council LearnEnglish cung cấp tài nguyên theo chủ đề và trình độ. Sự khác biệt này giải thích vì sao một hệ thống có thể đáp ứng tốt một nhu cầu nhưng vẫn cần được bổ sung khi áp dụng vào bối cảnh khác.

Khoảng trống thứ nhất liên quan đến sự kết nối giữa kiến thức và hoạt động. Công cụ ôn tập hoặc tổ chức câu hỏi không tự bảo đảm nội dung có quan hệ kế thừa; tài nguyên ngữ pháp có cấu trúc cũng chưa tự tạo thành quá trình học gắn với lớp học và tương tác giữa người dùng. Vì vậy, cần xem xét toàn bộ chuỗi từ tiếp cận lý thuyết, luyện tập đến đánh giá, thay vì chỉ đối chiếu số lượng chức năng.

Khoảng trống thứ hai liên quan đến đối tượng người học Việt Nam. Hỗ trợ bằng tiếng Việt không chỉ là chuyển ngữ giao diện, mà còn bao gồm cách giải thích quy tắc, lựa chọn ví dụ và trình bày sự khác biệt giữa các cấu trúc. Tuy nhiên, mức độ hữu ích của cách hỗ trợ này vẫn cần được kiểm chứng với người học thực tế.

Khoảng trống thứ ba là mối liên hệ giữa học tập cá nhân, quản lý lớp học và đối kháng. Trong phạm vi tài liệu đã khảo sát, chưa có đủ bằng chứng để xác nhận một hệ thống đáp ứng đầy đủ tổ hợp yêu cầu này theo đúng bối cảnh của đề tài. Đây là khoảng trống của phạm vi khảo sát, không phải khẳng định rằng thị trường hoàn toàn chưa có giải pháp tương ứng.

2.7 Kết luận

Kết quả phân tích cho thấy bài toán của Grammar Quest cần được đặt ở sự liên kết giữa nội dung ngữ pháp có cấu trúc, khả năng hỗ trợ người Việt và các hình thức học tập cá nhân, lớp học, đối kháng trực tuyến. Giá trị cần xem xét không chỉ là sự hiện diện của từng chức năng, mà còn là cách các chức năng phối hợp trong cùng quá trình học.

Từ đó, bài toán nghiên cứu được xác lập là xây dựng một nền tảng đáp ứng tổ hợp yêu cầu trên, đồng thời duy trì tính đúng đắn của dữ liệu và kết quả khi nhiều người dùng tương tác. Những nhận định về khoảng trống tạo cơ sở cho việc phát biểu yêu cầu ở chương tiếp theo; chúng chưa được xem là bằng chứng rằng Grammar Quest có hiệu quả học tập hoặc chất lượng tổng thể vượt trội so với các hệ thống đã khảo sát.

Chương 3

PHÁT BIỂU BÀI TOÁN

Grammar Quest là hệ thống web hỗ trợ học ngữ pháp tiếng Anh, kết hợp lộ trình tự học, quản lý lớp và thi đấu PvP. Bài toán đặt ra là tổ chức các hoạt động này trên cùng nền tảng, đồng thời bảo đảm đúng quyền truy cập, nội dung đề và kết quả khi nhiều người tương tác qua mạng. Chương này xác định yêu cầu, trạng thái, tình huống lỗi và những thuộc tính cần kiểm chứng của hệ thống.

3.1 Yêu cầu và phạm vi phục vụ

Hệ thống phục vụ ba nhóm người dùng: học viên, giáo viên và quản trị viên. Mỗi nhóm sử dụng một không gian làm việc riêng nhưng dùng chung tài khoản và cơ chế xác thực. Quyền phải được kiểm tra tại máy chủ; việc thay đổi URL, mã tài nguyên hoặc vai trò trong dữ liệu gửi lên không được tạo quyền ngoài phạm vi cho phép.

Bảng 3.1. Đối tượng sử dụng và chức năng chính.

Đối tượng	**Chức năng chính**	**Giới hạn quyền**
Học viên	Học theo lộ trình, luyện tập, làm bài tập và kiểm tra, tham gia PvP, xem kết quả	Chỉ truy cập nội dung đã mở, lớp được phép tham gia và lượt làm của mình
Giáo viên	Tạo lớp, quản lý thành viên, soạn học liệu, giao bài, tổ chức kiểm tra, xem bảng điểm	Chỉ quản lý lớp và học liệu thuộc quyền sở hữu
Quản trị viên	Quản lý tài khoản, xử lý báo cáo, hỗ trợ người dùng, xem nhật ký và giám sát	Thao tác quản trị được xác thực, kiểm tra quyền và lưu dấu vết phù hợp

Về tự học, nội dung được tổ chức theo cấp độ khóa học, module và bài học. Bộ nội dung hiện tại có 5 cấp độ và 28 module. Học viên mới thuộc diện đánh giá đầu vào phải hoàn thành bước kiểm tra để xác định điểm bắt đầu; kết quả này không tự đánh dấu các bài học đã hoàn thành. Từ cấp độ bắt đầu, module được mở theo tiến độ; bài kiểm tra cuối cấp độ gồm 10 câu, yêu cầu tối thiểu 8 câu đúng để mở cấp tiếp theo.

Về luyện tập, học viên được ôn theo module hoặc tổng hợp trong phạm vi đã mở. Đề được lựa chọn và gắn với lượt làm; máy chủ kiểm tra đáp án rồi tính kết quả. Dữ liệu cần giữ kín trước khi nộp không được chứa đáp án đúng hoặc lời giải. Cấp độ khóa học được phân biệt với cấp tài khoản theo XP và xếp hạng ELO; tăng XP hoặc thắng PvP không đồng nghĩa với hoàn thành nội dung học.

Về lớp học, giáo viên tạo và quản lý lớp, công bố bài giao, đặt thời hạn và ngưỡng đạt. Học viên tham gia bằng mã lớp rồi thực hiện bài được giao. Hệ thống giữ đề đã giao và lịch sử lượt làm; điểm chính thức được suy từ lượt hoàn tất đủ điều kiện thời gian, lựa chọn kết quả tốt nhất trong các lượt hợp lệ. Mở lượt làm lại không được xóa điểm đã có. Với kiểm tra trực tiếp, đáp án phải thuộc phiên bản đề hiện hành và được gửi khi bài còn hiệu lực.

Về PvP, học viên có thể tạo phòng riêng, tham gia bằng mã hoặc tìm đối thủ qua hàng đợi. Máy chủ kiểm tra điều kiện tài khoản, ghép cặp, quản lý thời hạn và xác định kết quả cùng thay đổi ELO. Hệ thống còn cung cấp nhiệm vụ ngày, XP, thông báo, trao đổi hỗ trợ và công cụ xử lý vi phạm để hỗ trợ quá trình sử dụng.

Yêu cầu phi chức năng quan trọng là bảo mật, nhất quán dữ liệu và giới hạn tài nguyên. Tài khoản được xác thực bằng phiên HTTP; mật khẩu sử dụng BCrypt; REST có bảo vệ CSRF. Thao tác có nguy cơ cạnh tranh phải được kiểm soát bằng giao dịch, khóa hoặc ràng buộc dữ liệu.

Hệ thống hướng tới học và kiểm tra ngữ pháp trong các chức năng đã hiện thực; chưa đặt mục tiêu chứng nhận CEFR, giám sát thi bằng camera hoặc chấm bài viết tự do bằng trí tuệ nhân tạo. Hiệu quả giáo dục và năng lực vận hành ngoài Internet cần được đánh giá bằng nghiên cứu riêng.

3.2 Mô hình trạng thái, thành phần và định danh

Có thể mô hình hóa trạng thái tổng quát bằng $S = (D, K, W)$. Trong đó, D là dữ liệu đã lưu; K là trạng thái tạm như hàng đợi, cache và đăng ký theo dõi; W là tập kết nối đang hoạt động. Đây là mô hình phân tích, không có nghĩa toàn bộ cơ sở dữ liệu và RAM được chụp nguyên tử tại một thời điểm. Dữ liệu trên giao diện là phần trạng thái mà tài khoản được phép xem, do máy chủ lựa chọn theo quyền.

Nhóm dữ liệu bền vững gồm tài khoản, nội dung học, thành viên lớp, đề, đáp án đã lưu, kết quả, XP và nhật ký. Lộ trình mới dùng các bảng `study_*`; các bảng bài học và lượt làm truyền thống vẫn tồn tại để phục vụ những luồng liên quan. Trạng thái trong RAM gồm socket, hàng đợi tìm đối thủ, thông tin hoạt động và tham chiếu gói đề. Backend khởi động lại không làm mất dữ liệu đã commit trong MySQL, nhưng các cấu trúc RAM cần được thiết lập lại.

Các định danh phải được dùng đúng phạm vi. `userId` xác định tài khoản; phiên HTTP xác định ngữ cảnh đăng nhập; `socketId` chỉ một kết nối. `classId`, `examId`, `attemptId` và `roomId` xác định tài nguyên nhưng không thay thế kiểm tra quyền. Mã lớp hoặc mã phòng phục vụ thủ tục tham gia, không cấp quyền quản lý. Một tài khoản có thể mở nhiều tab nên số tài khoản trực tuyến không bằng số socket.

Trong giao thức, `requestId` liên hệ yêu cầu WebSocket với phản hồi. Nó khác với `requestKey` dùng để nhận diện yêu cầu tạo lượt lộ trình, `clientMessageId` của tin hỗ trợ và `rewardKey` trong sổ XP. Các khóa nghiệp vụ này hạn chế tác dụng lặp trong từng luồng đã hỗ trợ; chúng không tạo bảo đảm thực hiện đúng một lần cho mọi thông điệp.

`version`, `revision` và `state_version` cũng có ý nghĩa theo từng phân hệ, không phải một bộ đếm chung của toàn hệ thống.

Với lộ trình, lượt làm bắt đầu ở `IN_PROGRESS`. Lưu nháp chỉ cập nhật đáp án; nộp bài mới chấm và kết thúc lượt. Bài kiểm tra cuối cấp chuyển sang `PASSED` hoặc `FAILED`; các loại lượt còn lại chuyển sang `COMPLETED`. Nộp lại lượt đã kết thúc trả về trạng thái hiện có. Với kiểm tra trực tiếp, việc gửi đáp án phải khớp `version`; đề đã thay đổi hoặc bài đã nộp thì dữ liệu cũ không được tiếp tục áp dụng.

Với PvP, vòng đời chính gồm chờ hoặc xác nhận tham gia, đang thi đấu và kết thúc. Các mốc hiện tại gồm 15 giây xác nhận, 5 giây đếm ngược, 20 giây mỗi câu và tối đa 180 giây mỗi trận. Tạm dừng do mất hiện diện được quản lý bằng các trường riêng; việc nối lại phải còn trong điều kiện và cửa sổ thời gian của phòng. Máy chủ quyết định các mốc này, không dựa vào đồng hồ hoặc điểm tự khai của client.

3.3 Mô hình lỗi và điều kiện khôi phục

Client có thể mất kết nối khi backend vẫn xử lý; backend có thể dừng trong khi dữ liệu đã commit còn nguyên; cơ sở dữ liệu có thể chậm hoặc từ chối ghi dù socket vẫn mở. Vì vậy, "đã kết nối" không đồng nghĩa với "đã nộp bài", còn hết thời gian chờ không chứng minh thao tác chưa được thực hiện.

Trường hợp cần chú ý là mất mạng sau khi dữ liệu đã lưu nhưng trước khi phản hồi tới trình duyệt. Client chỉ biết chưa nhận kết quả; nếu gửi lại một thao tác ghi không có cơ chế chống lặp, tác dụng nghiệp vụ có thể bị nhân đôi. Cách khôi phục phù hợp là đọc lại lượt làm, bài nộp hoặc phòng theo định danh hợp lệ trước khi quyết định thao tác tiếp theo.

Bảng 3.2. Các tình huống lỗi và cách xử lý chính.

Tình huống	**Phản ứng cần có**
Phản hồi chậm, socket mất kết nối	Kết thúc yêu cầu chờ; kết nối lại theo chính sách và đọc trạng thái chính thức
Hết phiên hoặc tài khoản bị khóa	Thu hồi hoặc từ chối socket cũ; không tự nối lại để bỏ qua quyết định bảo mật
Vượt giới hạn kết nối/yêu cầu	Từ chối có kiểm soát, thông báo quá tải và lùi thời gian thử lại
Đề kiểm tra đã đổi	Từ chối đáp án theo `version` cũ; đồng bộ lại nội dung trước khi tiếp tục
Giao dịch thất bại	Không báo thành công giả; đối chiếu dữ liệu đã lưu trước khi thực hiện lại
Backend khởi động lại	Đọc dữ liệu đã commit; thiết lập lại theo dõi và tham gia lại hàng đợi nếu cần

Phía client giới hạn 100 yêu cầu chờ trên mỗi tab, thời gian chờ 15 giây và hủy các yêu cầu còn chờ khi socket đóng. Chính sách nối lại phân biệt lỗi mạng với hết phiên, bị

khóa và bị quản trị viên ngắt. Phía server kiểm tra heartbeat, thời gian không thao tác và thu hồi tài nguyên của kết nối. Những cơ chế này tạo giới hạn hữu hạn cho việc chờ và cấp phát, không bảo đảm mọi yêu cầu luôn thành công dưới tải tùy ý.

Khôi phục cũng phụ thuộc phân hệ. Lượt học chỉ khôi phục được phần đề và đáp án đã lưu; lượt đã hoàn tất không quay lại trạng thái đang làm. Kiểm tra trực tiếp phải còn thời hạn, chưa đóng và chưa nộp. Hàng đợi ghép trận ở RAM không được xem là đã phục hồi chỉ vì socket mở lại. Với phòng PvP, quyền tham gia và thời gian nối lại phải còn hiệu lực; dữ liệu phòng trong MySQL không thay thế toàn bộ trạng thái điều phối đã mất trong RAM.

3.4 Các bất biến và tiêu chí kiểm chứng

Các bất biến xác định những điều hệ thống phải bảo toàn khi thao tác bình thường, gửi lặp hoặc xảy ra lỗi. Trọng tâm là quyền truy cập, ngữ cảnh đề, kết quả chính thức và tài nguyên. Kiểm chứng cần quan sát dữ liệu thực tế thay vì chỉ xác nhận giao diện không bị lỗi.

Bảng 3.3. Bất biến cốt lõi và tình huống kiểm chứng.

Bất biến	**Tình huống cần kiểm tra**
Danh tính và quyền do server xác định	Sửa `userId`, `classId` hoặc `attemptId`; giữ socket cũ sau khi đăng xuất hay khóa tài khoản
Điểm do máy chủ chấm, không lộ đáp án trước hạn	Gửi điểm giả, đáp án sai kiểu; kiểm tra dữ liệu nhận trước khi nộp
Nội dung thuộc đúng lượt và phiên bản	Sửa ngân hàng sau khi giao bài; gửi đáp án theo đề kiểm tra đã đổi
Thao tác lặp không nhân đôi tác dụng trong phạm vi đã định nghĩa	Tạo lượt cùng `requestKey`; nộp lại; nhận thưởng cùng `rewardKey`
Lịch sử và điểm chính thức nhất quán	Mở lượt làm lại sau khi có điểm; nộp quá hạn; so sánh lượt đầu, gần nhất và tốt nhất hợp lệ
Trạng thái kết thúc và tài nguyên được bảo vệ	Gửi đáp án sau khi kết thúc; hai tab cùng tìm trận; mở/đóng nhiều socket và kiểm tra thu hồi

Test chống lặp cần kiểm tra số bản ghi và giá trị kết quả; test giao dịch phải xét commit/rollback; test thu hồi cần so sánh bộ đếm trước và sau nhiều chu kỳ kết nối. Kiểm thử đơn vị xác nhận quy tắc, kiểm thử tích hợp xác nhận phối hợp HTTP/WebSocket/MySQL, còn kiểm thử trên mạng thực đánh giá tải và khả năng phục hồi. Ba mức này bổ sung cho nhau, không thay thế lẫn nhau.

Chương 4
THIẾT KẾ VÀ TRIỂN KHAI HỆ THỐNG

Chương này trình bày thiết kế và hiện thực hệ thống Grammar Quest, tập trung vào kiến trúc ứng dụng, tổ chức dữ liệu, xử lý nghiệp vụ và triển khai vận hành. Nội dung được phân tích từ mã nguồn và các migration của dự án. Trình tự trình bày đi từ thiết kế tổng thể đến các cơ chế cụ thể, qua đó làm rõ quan hệ giữa yêu cầu nghiệp vụ và giải pháp được hiện thực.

4.1 Thiết kế tổng thể hệ thống

4.1.1 Phân chia chức năng và trách nhiệm

Grammar Quest được thiết kế để phục vụ ba nhóm người dùng gồm học sinh, giáo viên và quản trị viên. Việc phân chia chức năng dựa trên trách nhiệm của từng nhóm trong quá trình học tập và quản lý nội dung. Học sinh sử dụng hệ thống để xác định trình độ, tiếp cận bài học, luyện tập, thực hiện bài giao và tham gia đối kháng. Giáo viên tổ chức lớp học, xây dựng học liệu, giao bài và theo dõi kết quả. Quản trị viên quản lý tài khoản, xét duyệt nội dung và xử lý các vấn đề phát sinh trong quá trình sử dụng.

Các vai trò cùng sử dụng một mô hình tài khoản, được phân biệt bằng thuộc tính `role` trong bảng `users`. Cách tổ chức này tập trung những thông tin chung như tên đăng nhập, mật khẩu và trạng thái tài khoản, đồng thời cho phép áp dụng quy tắc truy cập theo vai trò. Tuy nhiên, vai trò chỉ là điều kiện ban đầu. Với những tài nguyên cụ thể như lớp học, bản nháp hoặc phòng đấu, hệ thống còn cần kiểm tra quan hệ sở hữu và tham gia của người dùng.

Về mặt nghiệp vụ, hệ thống được chia thành các phân hệ tài khoản, học tập, lớp học, quản lý học liệu, đối kháng và hỗ trợ. Các phân hệ có liên hệ thông qua tài khoản và dữ liệu hoạt động, nhưng không có cùng đặc điểm thực thi. Chẳng hạn, bài học là nội dung có thể được truy cập nhiều lần, trong khi một lượt làm bài là hoạt động có trạng thái và kết quả riêng. Phòng đấu lại có trạng thái dùng chung cho hai người chơi, vì vậy cần cơ chế phối hợp cập nhật chặt chẽ hơn.

Sự phân chia này là cơ sở xác định ranh giới xử lý. Những thao tác thay đổi tiến độ, kết quả hoặc quyền truy cập phải được kiểm tra tại máy chủ. Giao diện chịu trách nhiệm trình bày và hướng dẫn thao tác, nhưng không quyết định tính hợp lệ cuối cùng của dữ liệu.

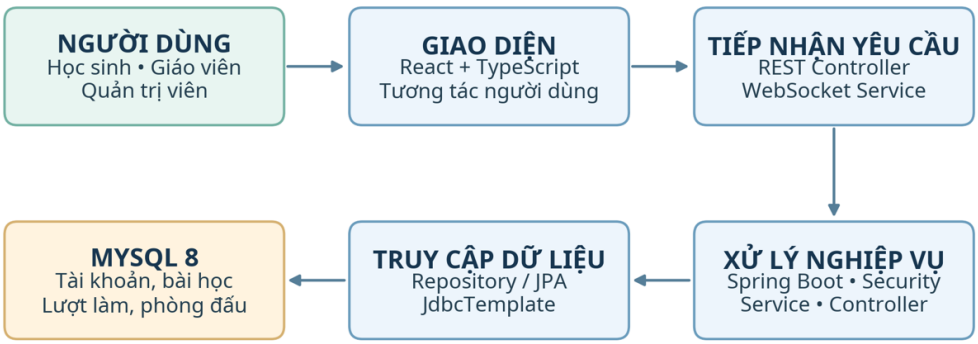
4.1.2 Kiến trúc client–server

Hệ thống sử dụng kiến trúc client–server với ba thành phần chính: ứng dụng giao diện, máy chủ xử lý và cơ sở dữ liệu. Phía giao diện được xây dựng bằng React và TypeScript, đảm nhiệm hiển thị nội dung, tiếp nhận tương tác và cập nhật trạng thái quan sát được. Máy chủ sử dụng Java 21 và Spring Boot để thực hiện xác thực, kiểm tra dữ liệu, xử lý nghiệp vụ và trao đổi thông tin với MySQL.

Việc lựa chọn TypeScript giúp mô tả cấu trúc dữ liệu trong mã giao diện và phát hiện một số sai khác kiểu dữ liệu trước khi chạy. Tuy nhiên, kiểm tra kiểu tại thời điểm biên dịch không thay thế kiểm tra dữ liệu nhận từ mạng. Máy chủ vẫn phải xác minh các giá trị như mã định danh, lựa chọn trả lời và trạng thái yêu cầu trước khi thực hiện nghiệp vụ.

Ở phía máy chủ, mã nguồn được tổ chức thành các nhóm controller, service, repository, entity, security và websocket. Cấu trúc này cho thấy sự phân chia trách nhiệm theo chức năng kỹ thuật. Controller tiếp nhận thao tác, service tổ chức một phần xử lý nghiệp vụ, repository hỗ trợ truy cập dữ liệu, còn các thành phần bảo mật và WebSocket xử lý những vấn đề chuyên biệt.

Tuy nhiên, kiến trúc hiện tại chưa tách biệt tuyệt đối các tầng. Một số controller, đặc biệt là `ArenaController`, trực tiếp chứa truy vấn `JdbcTemplate` và quy tắc nghiệp vụ. Vì vậy, mô hình thực tế là sự kết hợp giữa xử lý qua service và xử lý tập trung trong controller. Đặc điểm này cần được nhìn nhận khi đánh giá khả năng bảo trì: thay đổi quy tắc thi đấu có thể tác động trực tiếp đến lớp đang đồng thời đảm nhiệm tiếp nhận yêu cầu và truy cập dữ liệu.



Hình 4.1. Kiến trúc client–server của hệ thống Grammar Quest

4.1.3 Thiết kế giao tiếp giữa giao diện và máy chủ

Grammar Quest kết hợp REST API và WebSocket. REST được sử dụng cho các thao tác theo mô hình yêu cầu–phản hồi, còn WebSocket cung cấp kênh kết nối duy trì để trao đổi

thao tác và cập nhật trạng thái. Sự kết hợp này phù hợp với hệ thống có cả chức năng quản lý thông thường và hoạt động cần phản hồi liên tục như lớp học trực tiếp hoặc PvP.

Một yêu cầu quan trọng là các đường giao tiếp phải áp dụng cùng quy tắc nghiệp vụ. Nếu thao tác qua REST và WebSocket được hiện thực độc lập, những thay đổi về kiểm tra quyền hoặc điều kiện xử lý có thể chỉ được cập nhật ở một nơi. Khi đó, cùng một thao tác có nguy cơ tạo ra kết quả khác nhau tùy theo giao thức.

Trong phân hệ PvP, `SocketArenaService` tiếp nhận thao tác WebSocket và chuyển đến các phương thức tương ứng của `ArenaController`. Chẳng hạn, thao tác `ARENA_ANSWER` được chuyển đến phương thức `answer()`. Cách tổ chức này tái sử dụng điểm xử lý nghiệp vụ, qua đó giảm nguy cơ sai khác giữa hai đường truyền.

4.2 Thiết kế cơ sở dữ liệu

4.2.1 Tổ chức dữ liệu tài khoản và phân quyền

Bảng `users` lưu thông tin định danh và trạng thái tài khoản. Những thuộc tính như `username`, `password_hash`, `role` và `locked` lần lượt phục vụ nhận diện tài khoản, kiểm tra mật khẩu, phân quyền và kiểm soát khả năng truy cập. Tên đăng nhập được đặt ràng buộc duy nhất, giúp tránh tình trạng nhiều tài khoản cùng sử dụng một định danh đăng nhập.

Việc dùng chung bảng tài khoản tạo điều kiện để các hoạt động học tập, lớp học và thi đấu tham chiếu đến cùng một người dùng. Tuy nhiên, các khóa ngoại chỉ bảo đảm tài khoản được tham chiếu tồn tại; chúng không tự bảo đảm tài khoản có đúng vai trò nghiệp vụ. Chẳng hạn, một cột `teacher_id` tham chiếu đến `users` vẫn cần được kiểm tra tại ứng dụng để xác định người dùng phù hợp với trách nhiệm giáo viên.

Thuộc tính `placement_eligible` được bổ sung để xác định tài khoản thuộc diện cần kiểm tra đầu vào. Migration tương ứng giữ nguyên trạng thái miễn kiểm tra cho tài khoản đã có, trong khi luồng đăng ký quyết định áp dụng đối với tài khoản mới. Cách xử lý này cho thấy thay đổi chức năng đã được xem xét cùng ảnh hưởng đến dữ liệu đang tồn tại.

4.2.2 Tổ chức dữ liệu lộ trình học

Lộ trình học được biểu diễn bằng ba cấp dữ liệu gồm `study_levels`, `study_modules` và `study_lessons`. Mỗi cấp độ chứa các module, mỗi module chứa các bài học. Cấu trúc này cho phép tách đơn vị phân loại trình độ khỏi đơn vị tổ chức nội dung và bài học cụ thể.

Thứ tự được lưu bằng `ordinal_no`, thay vì suy ra từ mã định danh. Ràng buộc duy nhất trên cặp cấp độ–thứ tự và module–thứ tự hạn chế việc hai phần tử cùng chiếm một vị trí trong phạm vi cha. Nhờ đó, thay đổi cách sắp xếp không nhất thiết đòi hỏi thay đổi mã của bài học hoặc module.

Nội dung lý thuyết và câu hỏi của `study_lessons` được lưu trong `content_json` và `questions_json`. Hình thức này hỗ trợ biểu diễn nội dung có cấu trúc và lưu cùng bài học. Đổi lại, cơ sở dữ liệu không trực tiếp kiểm tra đầy đủ ý nghĩa của từng thành phần bên trong JSON. Trách nhiệm xác minh các trường bắt buộc, lựa chọn và đáp án thuộc về xử lý ứng dụng.

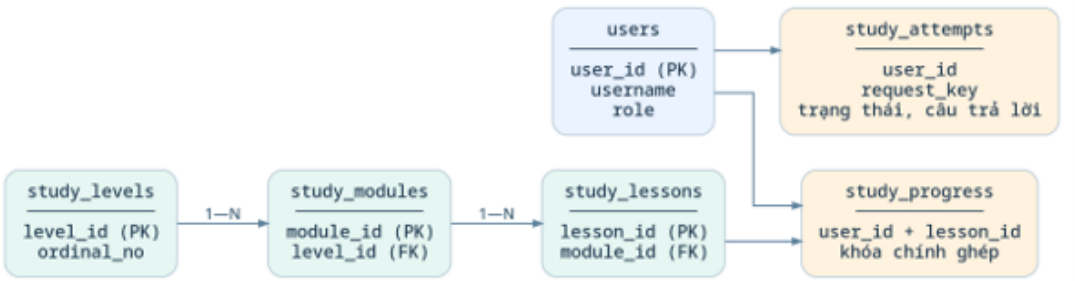
Tiến độ được lưu trong `study_progress` với khóa chính ghép (`user_id`, `lesson_id`). Thiết kế này thể hiện mỗi người học có tối đa một bản ghi tiến độ đối với một bài. Các thời điểm mở và hoàn thành được lưu trong cùng bản ghi, trong khi lịch sử làm bài được quản lý riêng.

4.2.3 Phân biệt tiến độ và lượt thực hiện

Một yêu cầu thiết kế là không đồng nhất tiến độ học tập với kết quả của một lần làm bài. Người học có thể thực hiện nhiều lượt đối với cùng nội dung, nhưng trạng thái hoàn thành của bài học là thông tin tổng hợp có ý nghĩa khác. Grammar Quest thể hiện sự phân biệt này thông qua các bảng tiến độ và các bảng lượt làm bài.

Trong `study_attempts`, mỗi lượt có mã riêng, người thực hiện, loại hoạt động, trạng thái, nội dung câu hỏi và câu trả lời. Cặp (`user_id`, `request_key`) được đặt ràng buộc duy nhất, tạo cơ sở nhận diện yêu cầu khởi tạo trùng trong phạm vi một tài khoản. Ràng buộc này hỗ trợ tính toàn vẹn, nhưng ứng dụng vẫn phải quyết định phản hồi thế nào khi gặp yêu cầu đã tồn tại.

Ngoài nhóm `study_*`, dự án còn có `lessons`, `questions`, `attempts` và `attempt_items`. Migration bổ sung lộ trình xác định việc giữ nguyên các nhóm dữ liệu có trước, bao gồm bài học, lượt làm bài, lớp học và đấu trường. Vì vậy, mô hình hiện tại tồn tại song song hai nhóm dữ liệu có liên quan về ý nghĩa nhưng khác cấu trúc và phạm vi sử dụng. Đây là đặc điểm cần được phản ánh khi xây dựng truy vấn, báo cáo và chức năng mở rộng.



Hình 4.2. Quan hệ dữ liệu học tập và tiến độ (mô hình rút gọn)

4.2.4 Dữ liệu lớp học, bài giao và kiểm tra

Bảng `classrooms` lưu lớp học và giáo viên phụ trách, còn `class_members` biểu diễn quan hệ giữa lớp với thành viên. Khóa chính ghép (`class_id`, `user_id`) ngăn cùng một tài

khoản được thêm nhiều lần vào một lớp. Thiết kế bảng liên kết cũng cho phép một người học tham gia nhiều lớp mà không phải lặp lại dữ liệu tài khoản.

Bài giao được lưu trong `assignments`, trong khi `assignment_attempts` liên kết bài giao với lượt làm bài. Cách tách này phân biệt yêu cầu do giáo viên tạo ra với hoạt động thực tế của học sinh. Một bài giao có thể nhận nhiều lượt làm, nhưng một lượt làm chỉ được liên kết với tối đa một bài giao nhờ ràng buộc duy nhất trên `attempt_id`.

Các hoạt động kiểm tra trực tiếp được quản lý bằng `class_exams` và `class_exam_submissions`. Dữ liệu kỳ kiểm tra chứa nội dung câu hỏi, thời hạn và trạng thái đóng; dữ liệu bài nộp chứa câu trả lời, số câu đã trả lời và trạng thái nộp của từng người học. Khóa chính (`exam_id`, `user_id`) xác định một bản ghi bài nộp cho mỗi người trong một kỳ kiểm tra.

Cách tổ chức này cho phép kiểm soát trạng thái bài nộp theo người tham gia, nhưng điều kiện tham gia lớp và thời hạn vẫn cần được xác minh trong nghiệp vụ.

4.2.5 Lưu giữ nội dung tại thời điểm sử dụng

Nội dung câu hỏi có thể được chỉnh sửa, trong khi kết quả hoạt động cần giữ nguyên ý nghĩa lịch sử. Nếu hệ thống chỉ lưu mã câu hỏi rồi đọc nội dung mới nhất khi xem lại, một thay đổi trong ngân hàng câu hỏi có thể khiến kết quả cũ không còn tương ứng với đề người học đã thực hiện.

Grammar Quest xử lý vấn đề này bằng cách lưu bản nội dung tại thời điểm sử dụng trong một số nghiệp vụ. `attempt_items` chứa nội dung, các lựa chọn, đáp án và giải thích của câu hỏi thuộc lượt làm bài. `arena_rounds` cũng lưu các dữ liệu tương ứng cho từng vòng đấu. Đối với bài giao, migration `V26__assignment_question_snapshot.sql` bổ sung trường `question_snapshot_json`.

Thiết kế này chấp nhận lặp dữ liệu để tăng khả năng tái hiện và đối chiếu. Việc lưu bản nội dung không có nghĩa đáp án được phép gửi ngay đến giao diện; dữ liệu lưu nội bộ và dữ liệu công bố cho người dùng vẫn phải được tách theo trạng thái hoạt động.

4.3 Thiết kế và hiện thực các nghiệp vụ chính

4.3.1 Xác thực và kiểm soát truy cập

Hệ thống sử dụng Spring Security và BCrypt để xử lý bảo mật tài khoản. Mật khẩu được lưu dưới dạng băm trong `password_hash`. Khi xác thực, máy chủ sử dụng thông tin tài khoản và trạng thái khóa để quyết định khả năng đăng nhập. Ngữ cảnh xác thực được duy trì thông qua phiên làm việc.

Cấu hình bảo mật có sử dụng cơ chế CSRF và cấu hình origin cho yêu cầu từ trình duyệt. Các cơ chế này cần được phối hợp với cách giao diện gửi yêu cầu và cách ứng dụng được định tuyến khi triển khai.

Phân quyền được tổ chức ở cả giao diện và máy chủ. Tại giao diện, `Protected`, `TeacherOnly` và `AdminOnly` điều hướng theo trạng thái đăng nhập và vai trò. Những thành phần này giúp người dùng tiếp cận đúng khu vực chức năng, nhưng không phải ranh giới bảo mật cuối cùng. Yêu cầu gửi trực tiếp đến máy chủ vẫn phải được kiểm tra độc lập.

Ở mức tài nguyên, hệ thống cần xác định người dùng có quan hệ phù hợp với đối tượng đang thao tác. Trong PvP, điều kiện này được thể hiện bằng kiểm tra người tham gia phòng. Đối với lớp học và học liệu, nguyên tắc tương ứng là kiểm tra thành viên, giáo viên phụ trách hoặc tác giả theo từng nghiệp vụ.

4.3.2 Kiểm tra đầu vào và điều hướng học tập

Kiểm tra đầu vào được tổ chức như một hoạt động có trạng thái, thay vì chỉ là một trang câu hỏi. Các bảng `study_entry_profiles` và `study_entry_attempts` lưu hồ sơ và lần thực hiện đánh giá của người học, còn `study_entry_questions` lưu câu hỏi theo cấp độ.

Ở giao diện, luồng truy cập xét đến vai trò học sinh, thuộc tính `placementEligible` và thông tin cho biết đánh giá còn bắt buộc hay không. Khi đang tải hoặc không tải được dữ liệu, giao diện hiển thị trạng thái tương ứng trước khi quyết định điều hướng. Điều này tránh kết luận quá sớm rằng người học được phép bỏ qua đánh giá.

Việc kiểm tra đầu vào được tách khỏi tiến độ bài học giúp hệ thống phân biệt trạng thái xác định điểm bắt đầu với trạng thái học tập về sau. Tuy nhiên, độ chính xác của việc xếp trình độ phụ thuộc vào nội dung câu hỏi và quy tắc đánh giá; cấu trúc phần mềm tự nó chưa chứng minh người học đã được phân loại phù hợp.

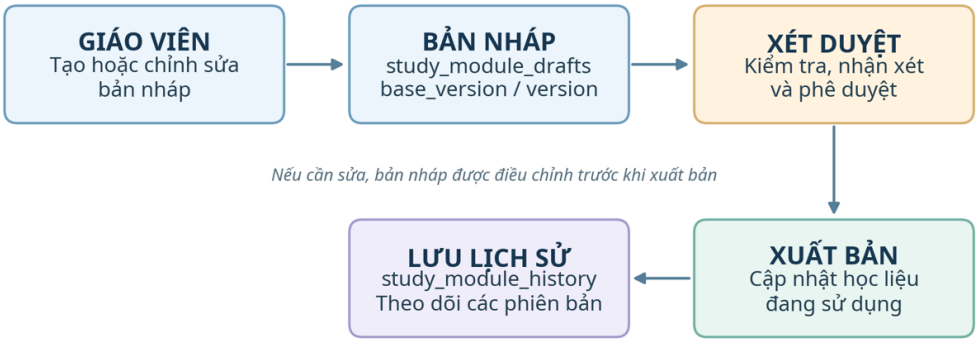
4.3.3 Biên soạn, xét duyệt và xuất bản học liệu

Quản lý học liệu được thiết kế theo hướng tách nội dung đang sử dụng khỏi nội dung được đề xuất. Bảng `study_module_drafts` lưu bản nháp, tác giả, trạng thái, người xét duyệt, nhận xét và dữ liệu đề xuất. Nhờ vậy, việc biên soạn không trực tiếp làm thay đổi nội dung mà học sinh đang truy cập.

Các trường `base_version` và `version` lưu thông tin phiên bản phục vụ đối chiếu. Về mặt thiết kế, thông tin này cần thiết khi nội dung gốc thay đổi sau thời điểm bản nháp được tạo. Việc phát hiện và xử lý xung đột cụ thể thuộc trách nhiệm của nghiệp vụ quản lý nội dung.

Bảng `study_module_history` lưu nội dung theo phiên bản cùng người thực hiện thay đổi. Lịch sử tạo cơ sở xem lại và xây dựng bản nháp khôi phục từ trạng thái trước. Đồng thời, thuộc tính `archived` của module và bài học cho phép ngừng sử dụng nội dung mà vẫn giữ các tham chiếu liên quan đến tiến độ.

Các cấu trúc này thể hiện quản lý học liệu đã được xem xét như một vòng đời nội dung, thay vì chỉ gồm thao tác thêm, sửa và xóa.



Hình 4.3. Vòng đời biên soạn, xét duyệt và lưu phiên bản học liệu

4.3.4 Tổ chức trạng thái và xử lý đối kháng PvP

PvP có yêu cầu đặc biệt vì hai người chơi cùng tác động đến một phòng đấu. Trạng thái chính thức được lưu tại máy chủ, bao gồm người tham gia, vòng hiện tại, thời hạn và điểm số. Giao diện gửi ý định thao tác, còn máy chủ quyết định thao tác có được chấp nhận hay không.

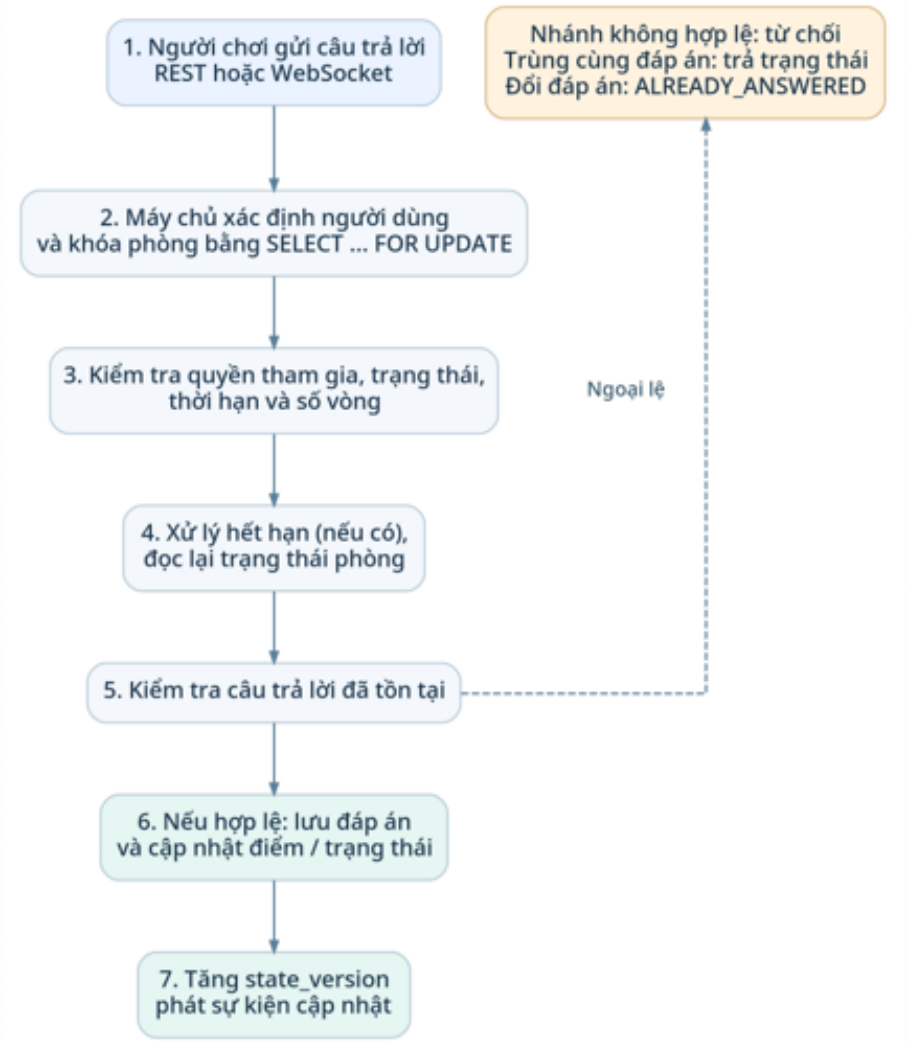
Phương thức `answer()` trong `ArenaController` thể hiện chuỗi kiểm tra trước khi ghi câu trả lời. Máy chủ xác định người dùng từ ngữ cảnh xác thực, lấy và khóa phòng, kiểm tra quyền tham gia, trạng thái tạm dừng, thời gian đếm ngược và số vòng được gửi. Sau khi xử lý điều kiện hết hạn, trạng thái phòng được đọc lại để xác nhận trận vẫn đang diễn ra và câu trả lời còn thuộc vòng hiện tại.

Việc kiểm tra lại có ý nghĩa vì xử lý thời gian có thể làm thay đổi trạng thái ngay trong quá trình tiếp nhận yêu cầu. Một câu trả lời mang số vòng đúng ở bước đầu vẫn có thể không còn hợp lệ sau khi hệ thống xác định vòng đã hết giờ. Nếu bỏ qua bước xác minh sau chuyển trạng thái, dữ liệu có nguy cơ được ghi vào thời điểm không còn phù hợp.

Tính toàn vẹn khi có yêu cầu đồng thời được hỗ trợ bằng giao dịch và truy vấn `SELECT...FORUPDATE` đối với phòng đấu. Khi các đường cập nhật cùng tuân thủ quy ước khóa, thao tác trên cùng phòng được phối hợp thông qua cơ sở dữ liệu. Cơ chế này hạn chế việc hai yêu cầu cùng xử lý dựa trên một trạng thái cũ, nhưng hiệu lực toàn hệ thống vẫn phụ thuộc phạm vi giao dịch và sự nhất quán của các đường ghi.

Đối với câu trả lời đã tồn tại, hệ thống phân biệt gửi lại cùng lựa chọn với thay đổi đáp án. Nếu lựa chọn giống dữ liệu đã lưu, máy chủ trả về trạng thái hiện tại. Nếu người chơi gửi lựa chọn khác, yêu cầu bị từ chối bằng lỗi `ALREADY_ANSWERED`. Cách xử lý này phù hợp với tình huống truyền lại khi người dùng chưa nhận được phản hồi trước đó, đồng thời giữ quy tắc không đổi đáp án sau khi đã trả lời.

Sau khi ghi nhận, phiên bản trạng thái được tăng và thông báo thay đổi được phát ra.



Hình 4.4. Quy trình xử lý câu trả lời trong chế độ PvP

4.3.5 Đồng bộ trạng thái và quản lý thời gian

Hệ thống sử dụng `state_version` để đánh dấu các thay đổi của phòng đấu. Phía giao diện so sánh phiên bản khi tiếp nhận dữ liệu, qua đó hỗ trợ ngăn trạng thái cũ ghi đè trạng thái mới hơn trong cùng phòng. Đây là cơ chế cần thiết khi dữ liệu có thể đến từ nhiều lần yêu cầu hoặc nhiều thời điểm cập nhật.

Tuy nhiên, phiên bản chỉ phản ánh thứ tự trạng thái, không giải quyết toàn bộ vấn đề đồng bộ. Khi mất kết nối hoặc chuyển phòng, giao diện cần xác định lại đối tượng đang theo dõi và tiếp nhận trạng thái phù hợp. Những trường hợp này phải được kiểm chứng trong bối cảnh vòng đời kết nối cụ thể.

Thời gian hiển thị trên giao diện được tính từ `deadlineAt` và đồng hồ máy khách. Trong khi đó, tính hợp lệ của câu trả lời được quyết định theo thời gian máy chủ. Cách phân chia này giữ quyền quyết định tại máy chủ, nhưng có thể dẫn đến chênh lệch giữa số giây người chơi nhìn thấy và thời hạn thực tế nếu đồng hồ thiết bị sai lệch.

Ngoài yêu cầu từ người chơi, dự án có xử lý định kỳ để thúc đẩy các trận đang hoạt động. `SocketArenaService.advance()` truy vấn những phòng đang xác nhận hoặc thi đấu rồi gọi xử lý tiến triển. Nhờ đó, việc chuyển trạng thái không hoàn toàn phụ thuộc vào thao tác mới từ phía người dùng. Tuy nhiên, thời điểm hoàn tất chuyển trạng thái còn chịu ảnh hưởng của chu kỳ kiểm tra và thời gian thực thi.

4.3.6 Quản lý dữ liệu tạm thời của phòng đấu

`ArenaSnapshotStore` lưu gói câu hỏi theo phòng trong bộ nhớ và theo dõi những kết nối đang sử dụng. Nhiều kết nối trong cùng phòng có thể tham chiếu đến cùng gói; khi không còn kết nối tham chiếu, dữ liệu được loại bỏ khỏi bộ nhớ.

Cấu trúc câu hỏi tại thành phần này gồm số vòng, nội dung và danh sách lựa chọn, không chứa đáp án đúng. Cách giới hạn trường dữ liệu giúp giảm nguy cơ đưa thông tin không cần thiết vào gói phục vụ kết nối. Đáp án vẫn được sử dụng trong xử lý nội bộ để chấm điểm và công bố theo điều kiện nghiệp vụ.

Dữ liệu trong bộ nhớ cần được phân biệt với dữ liệu bền vững. Bộ nhớ phục vụ truy xuất và quản lý hoạt động trong tiến trình hiện tại, còn MySQL lưu trạng thái trận và câu trả lời. Khi khởi động lại máy chủ hoặc triển khai nhiều tiến trình, các liên kết trong bộ nhớ không tự được chia sẻ hay phục hồi. Đây là giới hạn cần xem xét khi mở rộng mô hình triển khai.

4.4 Hiện thực giao diện và tích hợp hệ thống

4.4.1 Tổ chức trạng thái giao diện

Giao diện được chia thành các trang theo nghiệp vụ, kết hợp các thành phần dùng chung và context quản lý trạng thái. Những thông tin có phạm vi sử dụng rộng như xác thực, thông báo, nhiệm vụ hằng ngày, kiểm tra đầu vào và kết nối hệ thống được tổ chức thông qua các provider tương ứng.

Cách tổ chức này giúp nhiều màn hình sử dụng cùng trạng thái mà không phải tự duy trì các bản sao độc lập. Tuy nhiên, việc dùng context cần đi cùng quy tắc cập nhật rõ ràng, đặc biệt khi thông tin có thể thay đổi do yêu cầu HTTP và thông báo WebSocket.

Các thành phần hiển thị trạng thái tải và lỗi được tái sử dụng để phân biệt chưa có dữ liệu với không tải được dữ liệu. Đây là yêu cầu quan trọng trong ứng dụng bất đồng bộ, bởi việc hiển thị trang rỗng có thể khiến người dùng hiểu nhầm rằng không tồn tại nội dung.

Một số trang như đấu trường, quản trị và quản lý nội dung được khai báo tải trì hoãn. Cách tổ chức này cho phép mã của các chức năng được nạp khi cần, thay vì đưa toàn bộ vào luồng khởi tạo giao diện.

4.4.2 Phân biệt thao tác giao diện với kết quả nghiệp vụ

Trong hoạt động mạng, nhấn nút không đồng nghĩa thao tác đã được máy chủ chấp nhận. Vì vậy, giao diện cần phân biệt lựa chọn vừa thực hiện, yêu cầu đang gửi và kết quả đã xác nhận. Đặc biệt với PvP, việc tự cộng điểm hoặc tự chuyển vòng trước phản hồi chính thức có thể tạo ra trạng thái hiển thị không phù hợp với dữ liệu máy chủ.

Thiết kế phù hợp là cho phép phản hồi tức thời ở mức trình bày, nhưng sử dụng kết quả máy chủ cho những thay đổi có ý nghĩa nghiệp vụ. Khi yêu cầu bị từ chối do hết giờ, sai vòng hoặc trận kết thúc, giao diện cần phản ánh trạng thái mới thay vì chỉ giữ lựa chọn cục bộ.

Nguyên tắc này cũng áp dụng cho quản lý học liệu và lớp học. Một bản nháp chỉ được xem là lưu thành công sau khi máy chủ xác nhận; một bài làm chỉ được xem là đã nộp khi trạng thái tương ứng đã được ghi nhận.

4.5 Triển khai ứng dụng

4.5.1 Chuẩn bị cấu hình và cơ sở dữ liệu

Môi trường chạy cần Java 21, phiên bản Node.js phù hợp với Vite và MySQL 8. Các thông số kết nối cơ sở dữ liệu, cổng máy chủ và origin của giao diện được cung cấp thông qua biến môi trường hoặc tệp cấu hình cục bộ.

Flyway quản lý việc áp dụng migration khi ứng dụng khởi động. Cấu hình JPA sử dụng `ddl-auto:validate`, đặt trách nhiệm thay đổi schema vào migration thay vì để Hibernate tự cập nhật. Sự kết hợp này giúp tách kiểm tra ánh xạ dữ liệu khỏi quá trình thay đổi cấu trúc.

Trình tự khởi tạo gồm chuẩn bị cơ sở dữ liệu, thiết lập kết nối và khởi động máy chủ để áp dụng các migration chưa có. Với môi trường đã vận hành, thay đổi cần được bổ sung bằng migration mới. Việc sửa migration đã chạy có thể làm sai khác lịch sử và gây khó khăn khi đối chiếu giữa các môi trường.

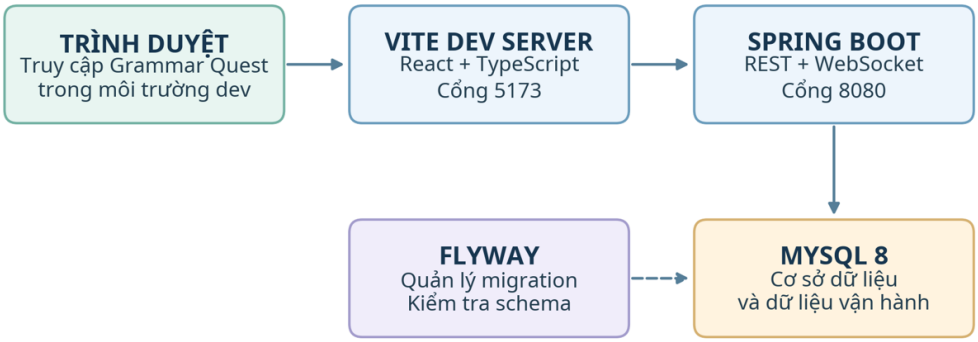
Ứng dụng có cơ chế tạo tài khoản mẫu để phục vụ phát triển và trình diễn. Dữ liệu mẫu cần được quản lý riêng với tài khoản sử dụng thực tế, tránh làm cho cấu hình phục vụ thử nghiệm trở thành cấu hình mặc định của môi trường công khai.

4.5.2 Triển khai trong môi trường phát triển

Trong cấu hình mặc định, máy chủ Spring Boot chạy tại cổng 8080 và Vite chạy tại cổng 5173. Vite chuyển tiếp các yêu cầu `/api` và `/ws` đến máy chủ, đồng thời hỗ trợ kết nối WebSocket.

Cơ chế proxy giúp giao diện sử dụng đường dẫn tương đối trong quá trình phát triển. Khi thay đổi cổng hoặc địa chỉ truy cập, cấu hình đích proxy và origin phải được điều chỉnh tương ứng. Nếu hai cấu hình không thống nhất, giao diện có thể tải được nhưng các yêu cầu xác thực hoặc WebSocket không hoạt động đúng.

Proxy của Vite chỉ phục vụ môi trường phát triển. Vì vậy, việc chạy thành công bằng dev server chưa đủ để kết luận bản đóng gói đã có cấu hình định tuyến phù hợp.



Hình 4.5. Sơ đồ kết nối các thành phần trong môi trường phát triển

4.5.3 Đóng gói và bố trí môi trường vận hành

Phía giao diện được đóng gói thông qua kiểm tra TypeScript và build Vite, tạo thư mục `dist`. Phía máy chủ sử dụng Maven Wrapper để kiểm thử và đóng gói thành JAR. Hai đầu ra được triển khai với trách nhiệm khác nhau: tài nguyên giao diện được phục vụ như nội dung tĩnh, còn JAR chạy như tiến trình ứng dụng.

Trong môi trường vận hành, cần có cơ chế định tuyến API và WebSocket đến máy chủ Spring Boot. Đường WebSocket phải hỗ trợ nâng cấp kết nối, còn cấu hình origin và phiên đăng nhập phải phù hợp với địa chỉ người dùng sử dụng. Những yêu cầu này xuất phát trực tiếp từ cách ứng dụng kết hợp giao diện tĩnh với máy chủ có phiên xác thực.

Hoàn thành build chỉ xác nhận phần mềm có thể được đóng gói trong điều kiện tương ứng. Khả năng vận hành cần được xác minh sau khi bố trí đầy đủ giao diện, máy chủ và cơ sở dữ liệu. Trong phạm vi tài liệu đã khảo sát, chưa có đủ bằng chứng xác nhận một môi trường công khai cụ thể đã được triển khai và đánh giá thành công.

4.5.4 Xác minh sau triển khai

Kiểm tra sau triển khai cần bắt đầu từ những chức năng nền tảng như đăng nhập, duy trì phiên và truy cập đúng vai trò. Tiếp theo là đọc học liệu, thực hiện bài làm và đối chiếu kết quả đã lưu. Các hoạt động lớp học và xét duyệt nội dung cần được kiểm tra bằng những tài khoản có trách nhiệm khác nhau để xác minh cả luồng hợp lệ và luồng bị từ chối.

Đối với WebSocket, cần kiểm tra thiết lập kết nối, nhận cập nhật và hành vi sau gián đoạn. Với PvP, ít nhất hai phiên người dùng phải được sử dụng để quan sát cùng phòng đấu. Những tình huống trả lời đồng thời, gửi lại câu trả lời và hết giờ cần được đối chiếu với trạng thái cuối cùng trong cơ sở dữ liệu.

Kho mã nguồn có các kiểm thử như `ArenaClockTest`, `SocketArenaServiceTest`, `ArenaMatchmakerTest` và `ArenaSnapshotStoreTest`. Tuy nhiên, sự tồn tại của các lớp kiểm thử không đồng nghĩa chúng đã vượt qua trong mọi môi trường. Một số kiểm thử tích hợp có điều kiện chạy riêng, nên kết quả thực nghiệm cần ghi rõ cấu hình và phạm vi đã thực hiện.

4.6 Nhận xét về thiết kế và triển khai

Thiết kế của Grammar Quest kết hợp giao tiếp REST và WebSocket để phục vụ các nghiệp vụ học tập, quản lý lớp học và thi đấu trực tuyến. Máy chủ chịu trách nhiệm xác thực, kiểm tra quyền, chấm điểm và quyết định trạng thái nghiệp vụ; giao diện tiếp nhận thao tác và hiển thị kết quả được xác nhận. Cách phân chia này giúp duy trì một nguồn trạng thái chính thức, hạn chế sự phụ thuộc vào dữ liệu do máy khách cung cấp.

Ở tầng dữ liệu, khóa chính ghép, ràng buộc duy nhất và cơ chế lưu nội dung câu hỏi tại thời điểm sử dụng hỗ trợ bảo vệ tính toàn vẹn và khả năng đối chiếu kết quả. Giao dịch và khóa được áp dụng để phối hợp những thao tác cập nhật đồng thời. Việc kiểm tra phiên bản, tách bản nháp khỏi nội dung đã xuất bản và lưu lịch sử thay đổi tạo cơ sở kiểm soát quá trình chỉnh sửa học liệu.

Đối với giao tiếp thời gian thực, hệ thống kết hợp cập nhật theo sự kiện với tác vụ định kỳ, sử dụng bộ đệm và so sánh snapshot để hạn chế xử lý, truyền dữ liệu lặp lại. Các giới hạn kết nối, yêu cầu đang chờ và tốc độ gửi giúp kiểm soát việc sử dụng tài nguyên. Tuy nhiên, hiệu quả của những cơ chế này phụ thuộc vào đặc điểm tải và cách phối hợp giữa các thành phần, nên cần được đánh giá bằng các kịch bản cụ thể.

Các cơ chế đã triển khai được kiểm chứng thông qua kiểm thử đơn vị, kiểm thử tích hợp với MySQL và các kịch bản trình duyệt. Chương 6 trình bày điều kiện thực hiện, kết quả và phạm vi suy luận tương ứng. Việc đánh giá phân biệt tính đúng đắn của nghiệp vụ, hiệu quả giảm thao tác dư thừa và khả năng kiểm soát hạn mức; kết quả của một nhóm kiểm thử không được sử dụng để thay thế cho kết luận về toàn bộ hiệu năng hệ thống.

Thiết kế hiện tại vẫn có những giới hạn cần tiếp tục hoàn thiện. Một số controller đảm nhiệm đồng thời việc tiếp nhận yêu cầu, xử lý nghiệp vụ và truy cập dữ liệu, làm tăng mức độ phụ thuộc khi thay đổi chức năng. Trạng thái kết nối, hàng đợi và bộ đệm nằm trong RAM của tiến trình nên chưa tự được chia sẻ hoặc phục hồi đầy đủ khi khởi động lại hay triển khai nhiều backend. Đồng hồ giao diện và hành vi kết nối lại cũng cần được đánh giá thêm trong điều kiện thiết bị sai giờ, mạng chậm hoặc gián đoạn kéo dài.

Nhìn chung, kiến trúc đã hình thành các cơ chế cần thiết để phối hợp chức năng và bảo vệ trạng thái nghiệp vụ trong phạm vi triển khai hiện tại. Việc mở rộng hệ thống cần đi cùng cải thiện phân chia trách nhiệm trong mã nguồn, bổ sung kiểm thử tình huống lỗi và đo hiệu năng dưới tải đại diện.

Chương 5

HIỆN THỰC VÀ ĐỘ TIN CẬY

Chương này chuyển từ mô hình kiến trúc sang cơ chế thực thi cụ thể trong mã nguồn Grammar Quest: cách máy chủ phân tách quyền sở hữu trạng thái giữa bộ nhớ khả biến (RAM) và cơ sở dữ liệu quan hệ (MySQL 8), cách điều phối ngân sách lập lịch đẩy trạng thái qua WebSocket song công, cách bảo toàn tính nguyên tử của giao dịch trước các tình huống cạnh tranh đa luồng, và cách đóng vòng đời của các phiên học tập, thi trực tuyến (Live Exam), đấu trường đối kháng (PvP Arena) cùng quy trình biên tập học liệu. Trọng tâm trình bày nằm ở ranh giới chịu trách nhiệm của từng lớp bảo vệ: cơ chế nào đảm bảo bất biến nào và cơ chế nào chủ đích không giải quyết bài toán nào.

5.1 Ownership và chính sách thực thi theo kênh phiên

Khác với các hệ thống chỉ giữ trạng thái trong một vòng lặp bộ nhớ đơn luồng, Grammar Quest phục vụ đồng thời ba vai trò (`LEARNER`, `TEACHER`, `ADMIN`) với yêu cầu bền vững cao đối với điểm kinh nghiệm (`total_xp`), bài nộp kiểm tra (`class_exam_submissions`), biên bản trận đấu (`arena_rooms`, `arena_rounds`) và bản nháp học liệu (`study_module_drafts`). Do đó, hệ thống phân tách rạch ròi hai miền sở hữu trạng thái:

1. Miền trạng thái bền vững có thẩm quyền (Authoritative Persistent State): Thuộc quyền sở hữu của MySQL 8 và được truy cập qua `JdbcTemplate` cùng các JPA repository trong phạm vi `@Transactional`. Mọi phán quyết làm thay đổi điểm số, quyền mở khóa lộ trình, kết quả chấm bài, biến thiên ELO hay phiên bản học liệu đều lấy hàng dữ liệu đã cam kết (committed row) trong MySQL làm nguồn chân lý duy nhất.

2. Miền trạng thái phiên và kênh theo dõi khả biến (Ephemeral Session & Watch State): Thuộc quyền sở hữu của bộ điều phối WebSocket `SystemSocketHandler`, bộ ghép trận `ArenaMatchmaker`, kho gói câu hỏi `ArenaSnapshotStore` và cổng kiểm soát sức chứa `ServerCapacity`. Tại `SystemSocketHandler`, trạng thái kết nối được phân mảnh thành các bảng `ConcurrentHashMap` độc lập thay vì dùng một khóa toàn cục: `userSessions` (`username->Set<UserSession>`), `sessionUsers` (`sessionId->username`), 5 bảng đăng ký theo dõi (`classWatches`, `examWatches`, `arenaWatches`, `workspaceWatches`, `supportWatches`) và tập đánh dấu bẩn `invalidatedUsers`.

Do đối tượng `WebSocketSession` mặc định không an toàn luồng khi nhiều luồng cùng ghi (ví dụ luồng phản hồi `requestId` giao thoa với luồng đẩy định kỳ `pushExamUpdates`), ngay tại `afterConnectionEstablished`, mỗi phiên vật lý được bọc bởi `Concurre`

ntWebSocketSessionDecorator với giới hạn thời gian gửi 5.000 ms và bộ đệm 512 KiB. Cơ chế này tuần tự hóa việc đẩy khung tin theo từng kết nối đơn lẻ: nếu một client bị nghẽn mạng khiến hàng đợi gửi vượt 512 KiB hoặc treo quá 5 giây, chỉ riêng phiên đó bị ngắt và dọn dẹp qua cleanupSession(), không gây nghẽn đầu hàng (head-of-line blocking) cho các người học khác.

Bảng 5.1. Các tình huống giao thoa đồng thời tiêu biểu và cơ chế phân xử trong Grammar Quest.

Tình huống giao thoa	Cơ chế điều phối trong mã nguồn	Kết quả khả kiến ở Client & Hệ thống
Hai luồng cùng ghi xuống 1 socket (phản hồi EXAM_ANSWER và nhịp pushExamUpdates)	ConcurrentWebSocketSessionDecorator tuần tự hóa hàng đợi gửi theo từng sessionId (5.000 ms, 512 KiB).	Khung JSON không bị trộn byte; nếu client ngừng đọc vượt 512 KiB, chỉ phiên đó bị đóng và thu hồi watch.
Học viên gửi liên tiếp EXAM_ANSWER cho cùng 1 câu hỏi trên một hoặc nhiều tab	AssignmentAnswerGuard.tryAcquire giữ khóa (username,examId,questionId) kết hợp isChanged() và SELECT ...FORUPDATE.	Đáp án trùng giá trị cũ được trả kết quả ngay từ RAM; gói gửi chồng lấn bị từ chối với mã 409 (ANSWER_IN_PROGRESS).
Giáo viên sửa đáp án/thang điểm (EXAM_EDIT) đúng lúc học viên nộp bài (EXAM_SUBMIT)	Cả hai hàm trong LiveExamService đều khóa bi quan hàng gốc SELECT*FROMclass_examsWHEREid=?FORUPDATE.	Hai giao dịch chạy tuần tự theo hàng class_exams; nếu EXAM_EDIT chạy sau, regradeExamSubmissions() tự chấm lại bài vừa nộp.
Quản trị viên đổi vai trò hoặc khóa tài khoản khi người dùng đang mở nhiều tab	syncAccountState phát ACCOUNT_SECURITY_UPDATE, đóng mọi socket của tài khoản và hủy HttpSession qua SocketSessionRegistry.	Mọi tab nhận sự kiện thu hồi quyền tức thì, đóng kết nối thời gian thực và mất hiệu lực phiên HTTP không chờ hết hạn cookie.
Hai đấu thủ hoàn tất câu cuối hoặc hết giờ PvP đồng thời từ hai luồng	ArenaController.locked khóa arena_roomsFORUPDATE, sau đó settle() khóa hai bản ghi users theo thứ tự min(id) rồi max(id).	Chỉ một luồng chuyển phòng sang FINISHED và tính ELO đúng 1 lần; luồng thứ hai trở thành thao tác rỗng (no-op) và không deadlock.

Ở cổng vào tiến trình, ServerCapacity đặt hàng rào kiểm soát sức chứa bằng Semaphore công bằng (FIFO) trước khi yêu cầu chạm tới bể kết nối HikariCP, thỏa mãn hệ ràng buộc:

$$\begin{cases} N_{\text{ws_global}} \le C_{\text{ws}} = 200, \quad N_{\text{ws}}(u) \le C_{\text{user}} = 5 \quad (\forall u \in \mathcal{U}) \\ N_{\text{http_active}} \le C_{\text{req}} = 32, \quad T_{\text{wait}} \le 1500\ \text{ms}, \quad P_{\text{hikari}}(16) < P_{\text{sched}}(8) + P_{\text{tomcat_min}}(8) \end{cases} \tag{5.1}$$

Khi một tài khoản vượt quá 5 kết nối WebSocket, phiên mới bị từ chối với mã 4003; khi toàn máy chủ chạm ngưỡng 200 phiên, kết nối bị đóng với mã 1013. Với các truy vấn HTTP đồng bộ (trừ tài nguyên tĩnh và /api/v1/auth/**), nếu không lấy được khe thực thi sau 1.500 ms, bộ lọc trả về ngay HTTP 503 Service Unavailable kèm header Retry-After: 3. Cần lưu ý rằng ServerCapacity là cơ chế kiểm soát ngưỡng nhận vào có giới hạn (bounded admission control) trên một nút máy chủ nhằm chống sập dây chuyền, không phải bộ cân bằng tải phân tán.

5.2 Ngân sách lập lịch và truyền trạng thái

Nếu mỗi thao tác ghi cơ sở dữ liệu đều lập tức kích hoạt truy vấn tổng hợp và tuần tự hóa JSON cho toàn bộ K kết nối đang quan sát, chi phí CPU sẽ tăng theo $\mathcal{O}(M \times K)$. Để giải quyết bài toán này, Grammar Quest vận hành cơ chế đẩy trạng thái theo nhịp lập lịch kết hợp gom nhóm tín hiệu làm mất hiệu lực (scheduled snapshot diffing & invalidation coalescing) trên bể luồng `realtimeTaskScheduler` (8 luồng tiền tố `realtime-` tại `RealtimeSchedulerConfig`) theo 4 tầng chu kỳ:

1. Nhịp 500 ms (`pushInvalidations`): Rút cạn nguyên tử tập `invalidatedUsers` và phát gói tin nhẹ `{"action": "DATA_INVALIDATED"}`. Mọi biến đổi dữ liệu của cùng một người dùng trong cửa sổ `500 ms` được gom tụ thành đúng một tín hiệu duy nhất. Phía trình duyệt, hook `useRealtimeRefresh` chặn rung (debounce) thêm `150 ms` và dùng cặp cờ `inFlight/queued` để đảm bảo tại mỗi thời điểm chỉ có tối đa một yêu cầu tải lại đang bay.

2. Nhịp 1.000 ms (`pushAccountUpdates`, `pushClassroomUpdates`, `pushExamUpdates`, `pushArenaUpdates`): Đồng bộ trạng thái tài khoản, lớp học, phòng thi và phòng đấu PvP. Trong mỗi nhịp, máy chủ khởi tạo bộ đệm cục bộ `Map<SnapshotKey,EncodedSnapshot>` với khóa `(username,resourceId,secondaryId,kind)`: dữ liệu chỉ được truy vấn và mã hóa JSON tối đa 1 lần cho mỗi bộ khóa và chỉ gửi xuống socket khi khác biệt cấu trúc với bản chụp trước đó (`!Objects.equals(encoded.snapshot(),watch.snapshot)`). Việc đặt `username` trong `SnapshotKey` bảo đảm cách ly quyền hạn tuyệt đối giữa giáo viên và học viên nhưng vẫn tái sử dụng kết quả mã hóa cho nhiều tab của cùng một tài khoản. Đầu mỗi nhịp `pushArenaUpdates`, máy chủ gọi `arenaSocket.advance(roomId)` cho mọi phòng đang mở, biến đồng hồ máy chủ thành nguồn thời gian có thẩm quyền (server-authoritative clock) tự động chuyển vòng ngay cả khi cả hai đấu thủ mất kết nối.

3. Nhịp 2.500 ms (`matchWaitingPlayers`, `pushWorkspaces`, `broadcastStats`): Quét ghép cặp đấu thủ trong `ArenaMatchmaker` theo biên độ cấp độ mở rộng dần theo thời gian chờ ($\Delta\text{Level} = 0$: ghép ngay; $\Delta = 1$: chờ ≥ 15 s; $\Delta = 2$: chờ ≥ 30 s; $\Delta > 2$: không ghép), đồng bộ không gian biên tập học liệu và phát bản tin giám sát `/topic/server-stats` chỉ chứa phần sự kiện chênh lệch `eventsDelta` (`event.id>lastBroadcastEventId`).

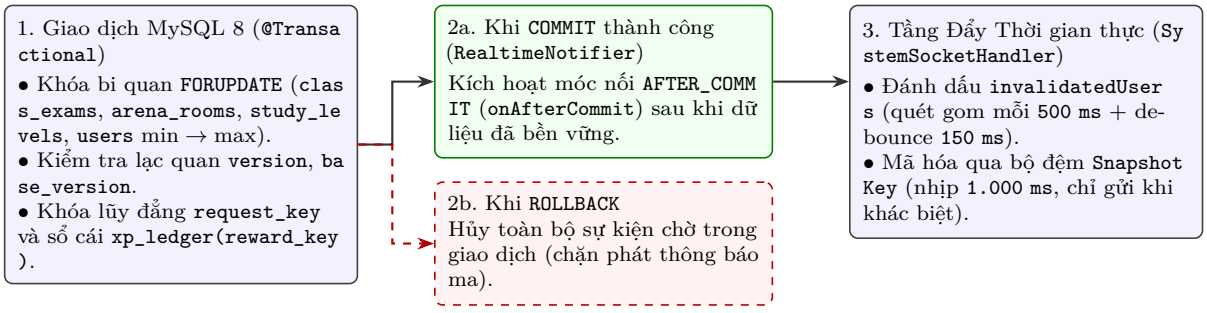
4. Nhịp 10.000 ms (`maintainConnections`): Phát gói `PING` và kiểm tra sức khỏe phiên. Các kết nối không hoạt động quá 120 giây bị ghi cảnh báo `WEBSOCKET_IDLE`; quá 30 giây không phản hồi `PONG` bị đóng cưỡng bức và dọn sạch khỏi mọi bảng watch; các bản ghi `ResourceWatch` quá 15 phút (`900.000 ms`) không làm mới cũng bị thu hồi tự động.

Bảng 5.2. Chính sách xử lý và tối ưu hóa theo từng loại dữ liệu bất đồng bộ trong Grammar Quest.

Loại dữ liệu / Kênh	Chu kỳ / Kích hoạt	Chiến lược tối ưu tại Server & Client	Rủi ro nếu thiếu điều phối
Tín hiệu làm mới tổng quát (`DATA_INVALIDATED`)	Sau `AFTER_COMMIT` + quét mỗi `500 ms`.	Gom nhiều thay đổi của cùng 1 `username` trong `500 ms` thành 1 cờ; client chặn rung `150 ms` và nối hàng `queued`.	Bão truy vấn HTTP (refetch storm) khi một giao dịch cập nhật nhiều bảng liên tiếp.
Ảnh chụp phòng thi & lớp học (`EXAM_DATA`, `CLASSROOM_DETAIL`)	Nhịp `1.000 ms` hoặc đẩy tức thời khi đổi trạng thái.	Bộ đệm `SnapshotKey` mã hóa JSON 1 lần/nhịp cho mỗi `(username,resourceId,kind)` và chỉ gửi khi khác bản chụp cũ.	Tuần tự hóa JSON lặp lại theo số tab và phát lại toàn bộ payload khi dữ liệu tĩnh.
Gói câu hỏi đấu trường PvP (`ArenaSnapshotStore`)	Cấp phát khi vào phòng (`acquire`), thu hồi khi rời (`release`).	Lưu `RoomPacket` bất biến dùng chung cho mọi phiên cùng `roomId` kèm bộ đếm `refCount`; chỉ nạp DB 1 lần và xóa khi refCount = 0.	Nhân bản câu hỏi trên RAM theo số lượng socket hoặc rò rỉ bộ nhớ sau khi trận đấu đóng.
Hàng đợi ghép trận PvP (`ARENA_QUEUE_STATUS`)	Sự kiện `ARENA_QUEUE_JOIN` + nhịp `2.500 ms`.	Giữ hàng đợi trong `ConcurrentHashMap` trên RAM; chỉ mở giao dịch DB khi hai người chơi thực sự khớp điều kiện tạo phòng.	Khóa bảng cơ sở dữ liệu liên tục mỗi giây chỉ để kiểm tra thời gian chờ.
Luồng giám sát máy chủ (`/topic/server-stats`)	Nhịp `2.500 ms` tại `ServerMonitorService`.	Bộ đệm vòng giới hạn 500 sự kiện; mỗi nhịp chỉ phát phần chênh lệch `eventsDelta` (id > lastBroadcastEventId, tối đa 50 mục).	Kích thước gói tin STOMP phình to tuyến tính theo thời gian vận hành máy chủ.

5.3 Giao dịch và tính nhất quán của persistence

Tầng lưu trữ của hệ thống được quản lý qua 36 kịch bản di trú Flyway (`V1`–`V36`), kết hợp lược đồ quan hệ chuẩn hóa với các cột tài liệu JSON (`content_json`, `questions_json`, `answers_json`, `payload_json`) để lưu trữ cấu trúc đề thi và học liệu đa hình.



Hình 5.1. Luồng bảo toàn tính nhất quán giao dịch kết hợp khóa bi quan, kiểm tra phiên bản lạc quan và đồng bộ thông báo `AFTER_COMMIT`.

Để ngăn ngừa mất cập nhật (lost update), đọc bẩn (dirty read) và khóa chết (deadlock), tầng persistence phối hợp bốn cơ chế (Hình 5.1):

1. Khóa bi quan mức hàng (SELECT...FORUPDATE) và thứ tự khóa định sẵn: Mọi thao tác làm bài, nộp bài hay sửa đề trong LiveExamService đều khóa hàng gốc class_exams trước khi đọc/ghi class_exam_submissions. Trong StudyManagementService, thao tác xuất bản học liệu lấy khóa danh mục catalogLock() (SELECTidFROM study_levelsORDERBYidFORUPDATE) trước khi khóa hàng bản nháp. Khi kết thúc trận PvP trong ArenaController.settle, hệ thống luôn khóa hai bản ghi User theo thứ tự định danh số học nghiêm ngặt min(hostId, guestId) trước rồi mới đến max(hostId, guestId), triệt tiêu hoàn toàn nguy cơ khóa chết vòng tròn (deadlock).

2. Kiểm soát đồng thời lạc quan (Optimistic Version Checking): Trong quy trình biên tập học liệu kéo dài nhiều phút, mỗi thao tác lưu hoặc duyệt trên study_module_drafts đều đối chiếu cột version (từ chối với mã 409STUDY_VERSION_CONFLICT nếu lệch). Khi xuất bản (publish), máy chủ kiểm tra thêm draft.base_version==study_modules.current_version để chặn bản nháp cũ ghi đè mù lên học phần vừa được người khác cập nhật (STUDY_BASE_VERSION_CONFLICT).

3. Khóa lũy đẳng (Idempotency Keys) và sổ cái bất biến (xp_ledger): Bảng study_attempts gắn ràng buộc duy nhất trên (user_id,request_key) để trả về chính lượt làm bài đã tạo nếu client gửi trùng yêu cầu ROADMAP_START. Mọi khoản cộng XP từ bài học, nhiệm vụ ngày và kiểm tra xếp lớp đều phải chèn qua bảng xp_ledger có khóa duy nhất UNIQUE(reward_key), đảm bảo mỗi phần thưởng chỉ được cộng đúng một lần.

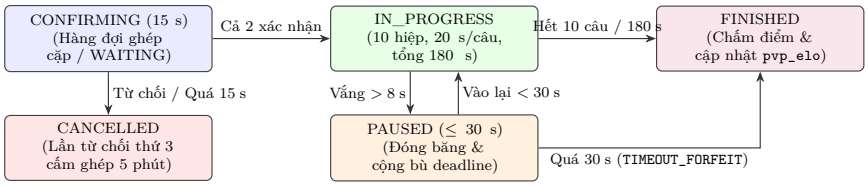
4. Đồng bộ thông báo sau cam kết (AFTER_COMMIT): RealtimeNotifier đăng ký mọi yêu cầu phát tín hiệu vào TransactionSynchronizationManager và @TransactionalEventListener(phase=TransactionPhase.AFTER_COMMIT). Chỉ khi MySQL xác nhận giao dịch đã COMMIT hoàn tất, tín hiệu mới được chuyển sang SystemSocketHandler; nếu giao dịch bị ROLLBACK, toàn bộ thông báo chờ tự động bị hủy bỏ.

5.4 Vòng đời và xử lý tình huống cạnh tranh

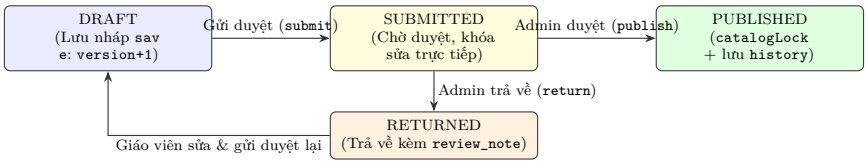
Mỗi miền nghiệp vụ trong Grammar Quest được điều khiển bởi một máy trạng thái hữu hạn chặt chẽ (Hình 5.2). Trong đấu trường PvP, kết quả trận đấu chỉ cập nhật pvp_elo (khởi điểm 1000, hệ số $K = 32$, 6 bậc từ BRONZE đến MASTER) và chuỗi thắng, tuyệt đối không cộng/trừ total_xp học tập. Khi hòa số câu đúng, người thắng được phân định bằng tổng thời gian trả lời tính theo micro-giây (host_time_us so với guest_time_us, câu sai phạt 20.000.000 μs). Trong quản trị lộ trình học, khi chèn hoặc đổi chỗ học phần, hệ thống dùng kỹ thuật hoán vị thứ tự âm hai pha (ordinal_no=-ordinal_no-1000 trước khi gán lại $1..N$) để tránh vi phạm UNIQUE(level_id,ordinal_no), đồng thời bật cờ preserveModuleAccess để việc thêm bài mới vào học phần trước không khóa ngược lại học phần sau mà học viên đang học dở.

Dưới đây là phân tích 4 tình huống cạnh tranh trọng yếu và tiêu chuẩn kiểm chứng (Oracle) tương ứng trong mã nguồn:

(a) Máy trạng thái vòng đời Phòng đấu PvP (arena_rooms):



(b) Máy trạng thái vòng đời Bản nháp Học liệu (study_module_drafts):



Hình 5.2. Máy trạng thái vòng đời của Phòng đấu PvP (`arena_rooms`) và Quy trình xuất bản học liệu (`study_module_drafts`).

- Sửa đề trực tiếp khi học viên đang chọn đáp án và nộp bài (`EXAM_EDIT` vs. `EXAM_ANSWER/EXAM_SUBMIT`): Khi giáo viên sửa đáp án đúng hoặc thang điểm của một đề đang mở cùng lúc 20 học viên đang làm bài, cả `saveAnswer`, `submit` và `editExam` trong `LiveExamService` đều xếp hàng tuần tự tại khóa `SELECT...FROMclass_examsWHEREid=?FORUPDATE`. Khi `editExam` thực thi, hệ thống tăng `revision=revision+1` và gọi `regradeExamSubmissions()` ngay trong cùng giao dịch để lọc bỏ câu trả lời của câu hỏi đã xóa đối với bài đang làm, đồng thời chấm lại toàn bộ bài `SUBMITTED` theo đáp án mới. Oracle: Không mất bất kỳ câu trả lời hợp lệ nào; mọi bài nộp trước hoặc sau thời điểm sửa đề đều hội tụ về cùng một điểm số theo `revision` mới nhất.

- Đổi vai trò hoặc khóa tài khoản khi người dùng mở nhiều tab: Khi quản trị viên khóa tài khoản hoặc hạ quyền từ `TEACHER` xuống `LEARNER`, `SystemSocketHandler.syncAccountState` phát sự kiện `ACCOUNT_SECURITY_UPDATE` tới mọi phiên của người dùng đó. Nếu bị khóa, toàn bộ socket bị đóng và `SocketSessionRegistry.closeHttpSessionsForUser` hủy ngay mọi `HttpSession`; nếu hạ quyền, `pruneIncompatibleWatches` xóa ngay người dùng khỏi `workspaceWatches` và cập nhật lại `SecurityContext`. Oracle: Tài khoản bị khóa mất quyền truy cập trên mọi tab tức thì (`401/403`); tài khoản bị hạ quyền không còn nhận được bản đẩy nào từ kênh đặc quyền cũ.

- Xuất bản học liệu cạnh tranh trên cùng một học phần (`StudyManagementService.publish`): Khi hai người biên tập cùng tạo bản nháp từ `base_version=1`, người thứ nhất xuất bản thành công sẽ nâng `study_modules.current_version` lên 2. Khi người thứ hai bấm xuất bản dưới khóa `catalogLock()`, phép kiểm tra `currentVersion!=baseVersion` lập tức ném lỗi `409STUDY_BASE_VERSION_CONFLICT` và rollback giao dịch; đồng thời `validateUniqueLessonIds` chặn mọi hành vi chèn `lessonId` trùng với học phần khác. Oracle: Bản nháp có `base_version` cũ bị từ chối `409`; lịch sử `study_module_history` tăng đơn điệu chặt và các lượt làm bài `study_attempts` đang mở vẫn giữ nguyên ảnh chụp câu hỏi ban đầu.

- Mất kết nối tạm thời và tạm dừng bù giờ trong trận đấu PvP (`applyPresenceState`): Trong `ArenaController`, nếu một đấu thủ mất kết nối WebSocket quá ngưỡng ân hạn 8 giây (`ABSENCE_THRESHOLD_SEC`), trận đấu tự động chuyển sang trạng thái tạm dừng tối đa 30 giây (`paused_until_at=now+30s`). Nếu người chơi kết nối lại trước 30 giây, hệ thống tính khoảng thời gian đã dừng Δt_{pause} và cộng bù chính xác vào cả `round_deadline_at` lẫn `match_deadline_at`; nếu quá 30 giây, nhịp `advance()` tự động xử thua `TIMEOUT_FORFEIT`. Oracle: Rớt mạng dưới 8 giây không gián đoạn trận; rớt mạng từ 8 đến 30 giây được đóng băng và bù đúng thời gian dừng khi vào lại; rớt mạng quá 30 giây tự động kết thúc trận ở phía server.

5.5 Bảo mật và mô hình triển khai

Hệ thống xác thực trong `SecurityConfig` sử dụng phiên máy chủ `HttpSession` (`JSESSIONID` với `HttpOnly = true`, `SameSite = Lax`, thời hạn 8 giờ), mật khẩu băm `BCrypt` và bảo vệ CSRF qua cookie `XSRF-TOKEN` / header `X-XSRF-TOKEN`. Để bịt lỗ hổng kết nối WebSocket vẫn sống sau khi phiên HTTP đã đăng xuất hoặc hết hạn, `SocketSessionRegistry` cài đặt cả `HttpSessionListener` và `HttpSessionIdListener`: mọi kết nối `/api/ws-system` và `/ws` đều được gắn với `httpSessionId` lúc bắt tay; khi phiên HTTP bị hủy (`sessionDestroyed`), toàn bộ socket liên kết bị đóng ngay lập tức với mã `4004` (`Session expired or logged out`), và khi đổi mã phiên chống cố định phiên (`sessionIdChanged`), ánh xạ được di trú nguyên tử sang phiên mới.

Tại tầng dịch vụ, danh tính người thực thi luôn lấy từ `Principal` của phiên đã xác thực chứ không bao giờ tin cậy `userId` trong payload client; các trường nhạy cảm (`correct`, `correctOption`, `acceptedAnswer`, `explanation`) bị cắt bỏ hoàn toàn khỏi JSON trước khi trả về cho học viên đang làm bài, và câu trả lời tự luận bị giới hạn cứng ở 500 ký tự. Để chống hành vi dùng script bắn phá đáp án (flooding) mà không làm ảnh hưởng đến các học sinh khác dùng chung IP NAT của phòng máy trường học, `AnswerRateLimiter` áp dụng cửa sổ trượt `1.000 ms` theo từng `username`: vượt quá 10 đáp án/giây sẽ kích hoạt khóa tạm thời 10 giây (`COOLDOWN_MS=10.000`) cho riêng tài khoản đó và đóng socket vi phạm với mã `4005`. Khi triển khai, giao diện React 19 + Vite 7 kết nối qua Reverse Proxy tới Spring Boot 3.3.5 (Java 21) + MySQL 8 thông qua một singleton WebSocket duy nhất trên mỗi tab (`systemSocket.ts`), tự động xếp hàng thông điệp khi đang kết nối và thử lại theo hàm mũ có nhiễu (`1.000 ms` đến `15.000 ms` cộng jitter `0-400 ms`).

Chương 6

THỰC NGHIỆM VÀ THẢO LUẬN

Chương này trình bày thiết kế thực nghiệm, hệ thống chỉ số đo lường, kết quả kiểm chứng và thảo luận kỹ thuật đối với Grammar Quest. Để đảm bảo tính trung thực học thuật và khả năng tái lập, báo cáo tách biệt rõ ba nguồn bằng chứng: (i) thực nghiệm đối chứng cơ chế đồng bộ thời gian thực và gom nhóm ảnh chụp (E_R), (ii) bộ kiểm chứng chức năng, giao dịch bất biến và giao diện tự động (E_V) gồm 132 phương thức kiểm thử JUnit 5 trên 35 lớp Java cùng 37 kịch bản kiểm chứng trình duyệt (`.mjs`), và (iii) thực nghiệm chịu tải tương tranh, kiểm soát sức chứa và giới hạn tốc độ (E_C) trên MySQL 8 cô lập.

6.1 Thiết kế thực nghiệm và metric

Hệ thống thực nghiệm được tổ chức nhằm trả lời bốn câu hỏi nghiên cứu (RQ1–RQ4):

- RQ1 (Hiệu quả đồng bộ & giảm tải dư thừa): Cơ chế gom nhóm `DATA_INVALIDATED` (`500 ms`), bộ đệm `SnapshotKey` (`1.000 ms`) và kho gói câu hỏi đếm tham chiếu `ArenaSnapshotStore` giảm chi phí tuần tự hóa, số khung tin WebSocket và số truy vấn DB như thế nào so với mô hình phát trực tiếp theo từng sự kiện/từng kết nối?
- RQ2 (Tính đúng đắn giao dịch & bất biến nghiệp vụ): Các cơ chế khóa bi quan `FOR UPDATE`, kiểm tra phiên bản lạc quan `version`/`base_version`, sổ cái lũy đẳng `xp_ledger`/`request_key` và đồng bộ `AFTER_COMMIT` có bảo toàn tuyệt đối dữ liệu trước các thao tác bất hợp lệ hoặc gửi trùng hay không?
- RQ3 (Chịu tải tương tranh & phân xử xung đột trực tiếp): Khi 20 luồng học viên cùng lưu/nộp bài đồng thời với lệnh sửa đề `EXAM_EDIT` của giáo viên, hệ thống có hội tụ đúng điểm số mà không mất cập nhật hay khóa chết hay không?
- RQ4 (Kiểm soát sức chứa & cô lập lạm dụng): Khi bùng phát kết nối vượt hạn mức (`ServerCapacity`) hoặc bắn phá đáp án vượt 10 lượt/giây (`AnswerRateLimiter`), hệ thống có chặn đúng đối tượng vi phạm và tự phục hồi hoàn toàn khi tải rút xuống hay không?

  Đối với tập E_R và E_C, báo cáo kiểm định hai cặp giả thuyết đối chứng:

- Giả thuyết H1: Giả thuyết không $H_{1,0}$ cho rằng bộ đệm `SnapshotKey`, so khớp `!snapshot.equals(watch.snapshot)`, gom nhóm `invalidatedUsers` và `ArenaSnapshotStore` không làm giảm số lần mã hóa JSON, số khung tin gửi đi hay số truy vấn nạp câu hỏi; ngược lại, giả thuyết đối $H_{1,1}$ khẳng định các cơ chế này giảm số khung tin gửi dư thừa về 0 trong các chu kỳ tĩnh, giảm số phép tuần tự hóa mỗi nhịp xuống $\mathcal{O}(U_{\text{watch}})$ và giảm số truy vấn nạp câu hỏi PvP xuống đúng 1 lần/phòng.

- Giả thuyết H2: Giả thuyết không $H_{2,0}$ cho rằng dưới tranh chấp 20 luồng học viên + 1 luồng giáo viên sửa đề hoặc bão 40–50 kết nối đồng thời, hệ thống bị mất cập nhật, sai lệch điểm hoặc rò rỉ giấy phép semaphore; ngược lại, giả thuyết đối $H_{2,1}$ khẳng định hệ thống bảo toàn 100% câu trả lời, hội tụ 100% điểm số sau chấm lại và phục hồi 100% dung lượng giấy phép sau khi giải phóng.

Với tập mẫu độ trễ hoặc chỉ số đo đã sắp thứ tự không giảm $x_{(1)} \le x_{(2)} \le \cdots \le x_{(n)}$, phân vị thứ $p \in (0, 100]$ (với $\lceil \cdot \rceil$ là hàm trần số nguyên) được tính theo công thức hạng gần nhất (nearest-rank percentile):

$$\mathrm{P}_p(X) = x_{\left(\lceil \frac{p}{100} \cdot n \rceil\right)} \tag{6.1}$$

Bảng 6.1. Định nghĩa hệ thống chỉ số thực nghiệm, phạm vi phản ánh hợp lệ và các diễn giải cần tránh.

Nhóm chỉ số (Metric)	Đơn vị & Nguồn thu	Phạm vi phản ánh hợp lệ	Diễn giải sai cần tránh
Tỷ lệ gom nhóm & khử trùng lặp ảnh chụp (`Coalescing&Diff`)	Số khung tin gửi thực tế / Số sự kiện phát sinh tại `SystemSocketHandler`.	Hiệu quả giảm chi phí CPU tuần tự hóa JSON và loại bỏ khung tin dư thừa ở tầng ứng dụng.	Không đồng nghĩa với giảm băng thông vật lý nếu dữ liệu biến đổi hoàn toàn ở mọi nhịp.
Số gói câu hỏi chia sẻ trên RAM (`packetCount`)	Số bản ghi `RoomPacket` hiện hành trong `ArenaSnapshotStore`.	Chứng minh tính chất chia sẻ $\mathcal{O}(1)$ theo phòng đấu và giải phóng sạch khi refCount = 0.	Không phải công cụ đo tổng dung lượng Heap JVM của toàn bộ tiến trình Spring Boot.
Độ hội tụ dữ liệu dưới tương tranh (`ConvergenceRate`)	Tỷ lệ bản ghi khớp tuyệt đối với Oracle sau giao dịch song song (%).	Tính đúng đắn của khóa `FORUPDATE`, `AssignmentAnswerGuard` và hàm `regradeExamSubmissions`.	Không dùng để ngoại suy thông lượng tối đa (TPS) trên cụm cơ sở dữ liệu phân tán nhiều nút.
Độ chính xác kiểm soát sức chứa (`PermitRecovery`)	Số kết nối/request được cấp (`acquired`), bị từ chối (`rejected`) và khe trống sau thu hồi.	Tính chặt chẽ của `Semaphore` công bằng và tính an toàn luồng trong `ServerCapacity`.	Kết quả chạy trên loopback (`127.0.0.1`) không bao gồm độ trễ bắt tay TLS hay suy hao gói trên WAN.

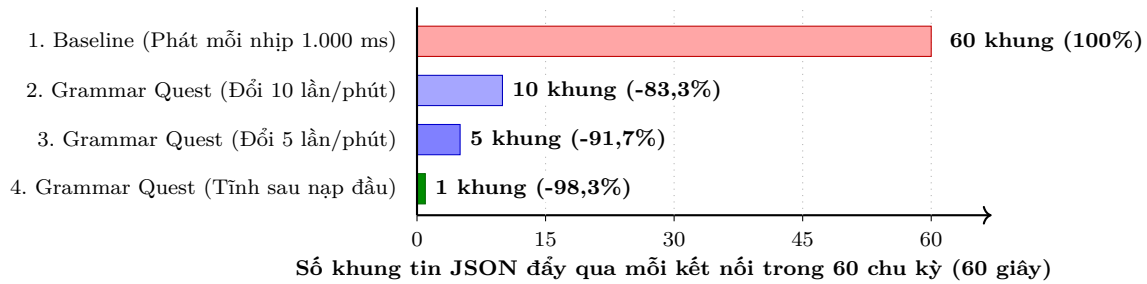
6.2 Kết quả đối chứng đồng bộ và gom nhóm cập nhật

Bảng 6.2 và Hình 6.1 tổng hợp kết quả đối chứng thực nghiệm của tập bằng chứng E_R (kiểm định qua `SystemSocketHandlerTest`, `ArenaSnapshotStoreTest` và `check-realtime-refresh.mjs`), trả lời trực tiếp câu hỏi RQ1.

Bên cạnh tầng đẩy dữ liệu, Bảng 6.3 trình bày kết quả kiểm định hai bộ điều phối trạng thái trên RAM là `ArenaMatchmaker` và `AssignmentAnswerGuard`, bác bỏ giả thuyết không $H_{1,0}$ và xác nhận giả thuyết đối $H_{1,1}$.

Bảng 6.2. Kết quả đối chứng định lượng giữa mô hình truyền trực tiếp và cơ chế gom nhóm / đệm ảnh chụp của Grammar Quest.

Kịch bản đối chứng	Mô hình cơ sở (Không gom)	Cơ chế trong Grammar Quest	Kết quả thực nghiệm & Bằng chứng
1. Đa tab trên cùng tài khoản theo dõi 1 phòng thi ($k = 5$ tab, đổi trạng thái 1 lần/nhịp)	Truy vấn dịch vụ và mã hóa JSON $k = 5$ lần riêng biệt trong cùng nhịp `1.000 ms`.	Bộ đệm `Map<SnapshotKey,EncodedSnapshot>` mã hóa theo khóa `(username,resourceId,secondaryId,kind)`.	1 lần truy vấn + 1 lần `writeValueAsString` cho cả 5 kết nối (giảm 80% chi phí tuần tự hóa ở ngưỡng 5 tab).
2. Theo dõi phòng thi / lớp học trong $T = 60$ nhịp liên tiếp khi không có thao tác ghi mới	Phát 60 khung tin `EXAM_DATA` / `CLASSROOM_DETAIL` cho mỗi phiên trong 1 phút.	Đối chiếu cấu trúc `!Objects.equals(encoded.snapshot(),watch.snapshot)` trước khi gửi.	1 khung tin lúc đăng ký đầu + 0 khung tin trong 59 nhịp tĩnh tiếp theo (giảm 98,3% khung tin đẩy qua socket).
3. Phát sinh M sự kiện thay đổi của 1 người dùng trong cửa sổ < 500 ms + < 150 ms ở Client	Phát M tín hiệu làm mới và kích hoạt M truy vấn HTTP `GET` tải lại trên trình duyệt.	Gom vào `Set<String> invalidatedUsers` (`500 ms`) + chặn rung `150 ms` và cờ `inFlight`/`queued` ở client.	M sự kiện gom thành 1 khung `DATA_INVALIDATED`; khi 5 tín hiệu đến lúc đang `inFlight`, client chỉ gọi tối đa 2 lượt tải.
4. Nạp danh sách 10 câu hỏi PvP cho 2 đấu thủ (mỗi người mở 1..2 tab)	Mỗi kết nối tự truy vấn `arena_rounds` và giữ bản sao riêng (2..4 truy vấn DB, 2..4 bản sao RAM).	`ArenaSnapshotStore` dùng chung 1 `RoomPacket` bất biến có `refCount`; chỉ đọc DB khi `packet==null`.	Đúng 1 truy vấn SQL khi người đầu tiên `acquire()`; packetCount = 1 trong trận và về 0 ngay khi người cuối `release()`.
5. Phát bản tin giám sát `/topic/server-stats` mỗi `2.500 ms` khi bộ đệm lưu 500 sự kiện	Mỗi nhịp phát lại toàn bộ 500 sự kiện lịch sử cho mọi tab quản trị viên.	Con trỏ `lastBroadcastEventId` chỉ trích xuất `eventsDelta` (id > lastBroadcastEventId, tối đa 50 mục).	Trong các nhịp không có cảnh báo mới, `eventsDelta` có 0 phần tử, giữ kích thước bản tin ổn định.



Hình 6.1. Đối chứng số khung tin WebSocket đẩy xuống mỗi kết nối trong 60 chu kỳ (`1.000 ms`/chu kỳ) giữa mô hình cơ sở và cơ chế `Diff-before-Send`.

Bảng 6.3. Kết quả thực nghiệm bộ ghép trận mở rộng biên độ cấp độ `ArenaMatchmaker` và bộ lọc đáp án `AssignmentAnswerGuard`.

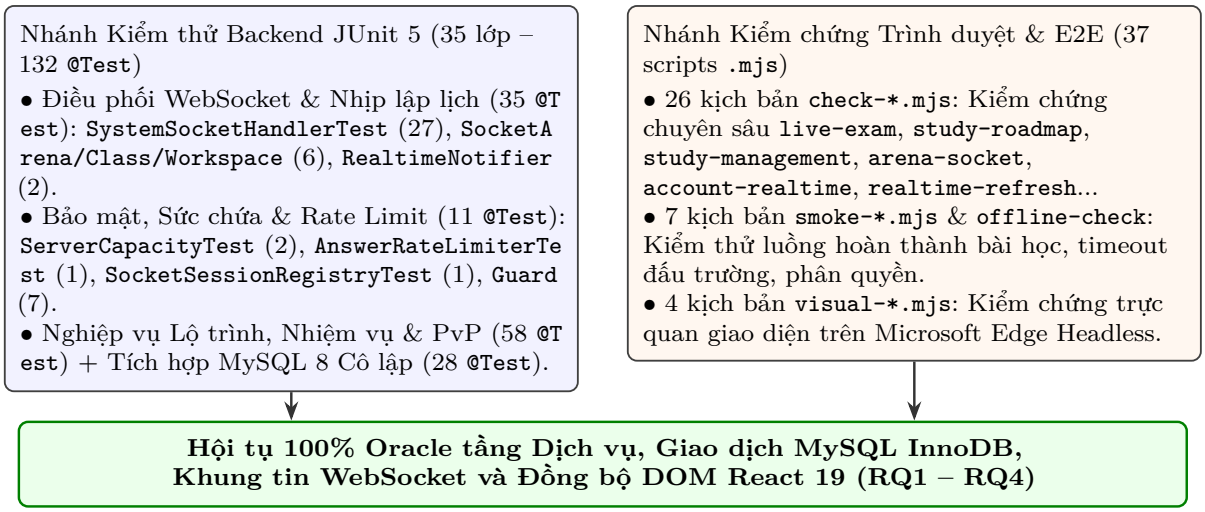
Bộ phận & Kịch bản	Tham số đầu vào	Hành vi thực thi tại RAM & DB	Kết quả quan sát & Khẳng định
`ArenaMatchmaker`: Ghép cặp cùng cấp độ (`sameLevel...`)	Người chơi `a` (Level 3) và `b` (Level 3) vào hàng đợi tại t_0.	Giữ trên RAM; tại `tick($t_0$)`, ΔLevel $= 0$ thỏa điều kiện ghép tức thì.	Cả `s1` và `s2` chuyển sang `MATCHED` ngay tại t_0; `factory.create("a","b")` gọi đúng 1 lần, hàng đợi về rỗng.
`ArenaMatchmaker`: Mở rộng biên độ theo thời gian chờ	Cặp (L_3, L_4) $(\Delta = 1)$; cặp (L_3, L_5) $(\Delta = 2)$; cặp (L_1, L_4) $(\Delta = 3)$.	Ngưỡng chờ theo độ lệch cấp: $\Delta = 0 \Rightarrow 0$ s; $\Delta = 1 \Rightarrow 15$ s; $\Delta = 2 \Rightarrow 30$ s; $\Delta \geq 3 \Rightarrow \infty$.	Cặp $\Delta = 1$ khớp tại $t_0 + 16$ s; cặp $\Delta = 2$ khớp tại $t_0 + 31$ s; cặp $\Delta = 3$ giữ `WAITING` sau 120 s và không bao giờ gọi DB.
`ArenaMatchmaker`: Chặn tài khoản bị cấm / vào trùng	Từ chối xác nhận trận 3 lần liên tiếp (pvpDeclineCount = 3) hoặc mở tab thứ 2 gọi `join`.	Kiểm tra `pvpBanUntil.isAfter(now)` và kiểm tra trùng `username` trên RAM.	Lần từ chối thứ 3 đặt pvpBanUntil = now + 300 s; lệnh `join` bị chặn với mã `MATCHMAKING_BANNED`, giữ nguyên `totalXp` (2500 XP).
`AssignmentAnswerGuard`: Khử trùng & chặn gửi chồng lấn	Gửi cùng đáp án `"A"` 2 lần liên tiếp, hoặc gửi gói 2 khi gói 1 đang giữ khóa `(user, examId,qId)`.	`tryAcquire` dùng `inFlight.add(key)`; `isChanged` đối chiếu `lastSavedAnswers.get(key)`.	Khi gói 1 đang xử lý, gói 2 nhận `false` (409); sau khi gói 1 lưu `"A"`, gói gửi lại `"A"` nhận `isChanged==false` và không chạm MySQL.

6.3 Kiểm chứng chức năng và tính nhất quán

Để trả lời câu hỏi RQ2, tập bằng chứng E_V thực thi kiểm chứng tự động trên hai nhánh (Hình 6.2): 132 phương thức `@Test` JUnit 5 ở phía máy chủ và 37 kịch bản kiểm chứng trình duyệt (`.mjs`) chạy trên Microsoft Edge Headless + Playwright. Nhằm bảo vệ môi trường dữ liệu (được cưỡng bức bởi `TestDatabaseGuardTest`), các bài kiểm thử tích hợp như `FullRealtimeIntegrationTest` và `StudyRoadmapIntegrationTest` luôn tự khởi tạo một schema MySQL dùng một lần khớp mẫu `grammar_quest_(it|study)_[0-9a-f]{12}_test`, chạy đủ 36 bản di trú Flyway trong `@BeforeAll` và xóa sạch bằng `DROPDATABASEIFEXISTS` trong `@AfterAll`.

Các kết quả kiểm chứng bất biến quan trọng nhất từ tập E_V bao gồm:

1. Tính toàn vẹn của lộ trình 5 cấp độ / 28 học phần / 120 bài học (`StudyRoadmapIntegrationTest`): Xác nhận người học mới chỉ mở khóa Cấp độ 1 và học phần đầu (`l1-m1`); mọi truy cập nhảy cóc sang bài `2.1`, học phần `l1-m2` hoặc chèn dữ liệu tiến độ rời rạc vào `l1-m3` khi chưa hoàn thành `l1-m2` đều bị chặn bởi `ApiException`. Trong khi làm bài (`IN_PROGRESS`), cây JSON gửi xuống học viên hoàn toàn không chứa `correctOption`, `correct` hay `explanation`. Bài kiểm tra tổng kết Cấp độ 1 (`LEVEL_TEST`) lấy 10 câu hỏi trải đều trên 4 học phần: đạt 7/10 câu trả về `FAILED` (Cấp độ 2 vẫn khóa); đạt 8/10 câu (80%) trả về `PASSED` và mở khóa Cấp độ 2; nếu thi lại đạt 0/10 câu, Cấp độ 2 vẫn giữ nguyên trạng thái đã mở.



Hình 6.2. Cấu trúc phân tầng của hệ thống kiểm chứng tự động (E_V và E_C) trong dự án Grammar Quest.

2. An toàn quy trình duyệt học liệu và bảo toàn quyền mở khóa (`StudyManagementValidationTest` & `managementWorkflow`): Xác nhận bản nháp đang soạn dở (chưa có câu hỏi hoặc trống nội dung lý thuyết) có thể lưu nháp (`save`) nhưng bị chặn khi gửi duyệt (`submit`); đồng nghiệp không thể xem hoặc xóa bản nháp riêng của giáo viên khác. Khi quản trị viên xuất bản một học phần mới chèn vào vị trí `ordinalNo=1`, học phần `l1-m1` lùi xuống vị trí 2 nhưng vẫn giữ mở khóa (`unlocked==true`) cho học viên đang học dở nhờ cờ `preserveModuleAccess`, đồng thời các lượt làm bài `study_attempts` đã tạo trước đó vẫn giữ nguyên ảnh chụp `questions_json` cũ.

3. Tính tách biệt của hệ thống ELO và kiểm soát câu hỏi tự luận (`PvpArenaFeaturesTest`, `LiveExamTextQuestionsTest`): Xác nhận 6 bậc xếp hạng từ `BRONZE` (950) đến `MASTER` (2100): hai đấu thủ cùng 1000 ELO thắng/thua ±16 điểm, người yếu (1000) thắng người mạnh (1400) nhận +29 ELO mà tuyệt đối không làm thay đổi `totalXp` học tập. Với câu hỏi tự luận (`TEXT/FILL_IN_BLANK`), hệ thống chuẩn hóa khoảng trắng, chấm không phân biệt hoa thường, chặn đáp án vượt 500 ký tự và tự động chấm lại toàn bộ bài nộp khi giáo viên cập nhật `acceptedAnswer`.

6.4 Tải tham chiếu, kiểm soát sức chứa và phạm vi suy luận

Mục này trình bày kết quả thực nghiệm của tập bằng chứng E_C (trả lời RQ3, RQ4 và kiểm định giả thuyết H2), bao gồm bài kiểm thử tương tranh đa luồng trên MySQL thật `LiveExamConcurrencyTest`, `ServerCapacityTest`, `AnswerRateLimiterTest` và `FullRealtimeIntegrationTest` (Bảng 6.4 và Bảng 6.5).

Về độ giá trị của thực nghiệm (threats to validity): (i) Độ giá trị nội tại đạt mức cao nhờ các bài kiểm thử tương tranh và tích hợp chạy trực tiếp trên MySQL 8 thật với 36 bản di trú Flyway và khóa hàng InnoDB thật thay vì dùng cơ sở dữ liệu giả lập trong bộ nhớ; (ii) Độ giá trị đo lường dựa trên các đại lượng bất biến xác định (số truy vấn SQL, số permit cấp phát/thu hồi, mã đóng WebSocket `4003/4004/4005`, điểm hội tụ `50%`), không bị nhiễu bởi dao động xung nhịp CPU; (iii) Độ giá trị ngoại tại có giới hạn rõ ràng vì các

Bảng 6.4. Tổng hợp kết quả thực nghiệm tương tranh đa luồng, kiểm soát sức chứa và giới hạn tốc độ (E_C).

Kịch bản (E_C)	Cấu hình & Điều kiện biên	Kết quả quan sát thực nghiệm	Đánh giá (H2)
1. Làm/nộp bài & sửa đề đồng thời (`LiveExamConcurrencyTest`)	20 học viên (20 luồng) + 1 giáo viên gọi `EXAM_EDIT` đổi đáp án câu 1 (A→B); 10 nộp ngay, 10 nộp sau sửa đề.	0 mất cập nhật; `revision` tăng 1 → 2; cả 20/20 bài nộp đều `SUBMITTED` với `correct_count=1/2`, `score_percent=50`.	Đạt 100% (Hội tụ `50%` nhờ `FORUPDATE`).
2. Bão kết nối WebSocket đa luồng (`ServerCapacityTest`)	50 yêu cầu bắt tay cho `"alice"` từ 12 luồng worker (maxSockets = 20, maxPerAccount = 3).	Chính xác 3 yêu cầu được cấp (`acquired=3`), 47 yêu cầu bị chặn (`rejected=47`); sau 3 lần `release`, bộ đếm về 0.	Đạt 100% (Không lọt vượt ngưỡng, không rò permit).
3. Bão 40 socket thật & Ngưỡng HTTP 503 (`FullRealtimeIT`)	Mở đồng thời 40 socket thật (maxPerAccount = 5); giữ kín 32 permit HTTP (wait = 1500 ms) rồi gọi REST.	Tối đa ≤ 5 socket mở, ≥ 35 socket đóng với mã `4003`; truy vấn REST nhận đúng `HTTP 503` (`Retry-After: 3`) và về `200` khi nhả permit.	Đạt 100% (Bảo vệ cả WS & HTTP, tự phục hồi).
4. Chống bắn phá đáp án theo tài khoản (`AnswerRateLimiter`)	`"alice"` gửi liên tiếp 11 đáp án trong < 1 s cùng IP với `"bob"` (MAX = 10/s, COOLDOWN = 10 s).	10 đáp án đầu hợp lệ; đáp án 11 bị chặn (`RATE_LIMITED`), socket đóng với mã `4005`; cùng lúc đó `"bob"` vẫn gửi đáp án bình thường.	Đạt 100% (Cô lập đúng tài khoản trên cùng IP).

bài kiểm thử chạy trên giao diện `loopback127.0.0.1`, do đó chứng minh tính đúng đắn của cơ chế điều phối đồng thời và kiểm soát sức chứa trên một nút máy chủ nhưng không thay thế cho phép đo độ trễ mạng diện rộng (WAN).

6.5 Thảo luận kết quả và ưu tiên cải tiến

Kết quả thực nghiệm từ hai chương cho thấy sự ổn định khả kiến ở người dùng là sản phẩm cộng hưởng của bốn lớp bảo vệ nối tiếp nhau: (1) Tầng trình duyệt gom giao tiếp vào một singleton WebSocket và chặn rung `150 ms` qua `useRealtimeRefresh` để chống giật giao diện; (2) Tầng cổng vào (`ServerCapacity`, `AnswerRateLimiter`, `SocketSessionRegistry`) từ chối sớm các kết nối vượt hạn mức (> 5 socket/tài khoản, > 32 request HTTP) và các luồng flood đáp án (> 10 lượt/giây) trước khi chúng làm cạn bể kết nối HikariCP (16 kết nối); (3) Tầng lập lịch thời gian thực (`SystemSocketHandler`, `ArenaSnapshotStore`) tách nhịp đẩy dữ liệu (`500 ms`, `1.000 ms`, `2.500 ms`) khỏi tần suất ghi giao dịch và dùng bộ đệm `SnapshotKey` để triệt tiêu truyền tải dư thừa; (4) Tầng giao dịch MySQL đảm bảo tính nguyên tử qua khóa `FORUPDATE` có thứ tự, kiểm tra phiên bản lạc quan và chỉ phát thông báo ở pha `AFTER_COMMIT`.

Trong vận hành thực tế, quản trị viên có thể chẩn đoán nhanh trạng thái máy chủ qua các bộ đếm của `ServerMonitorService`:

- Nếu `rejectedConnections` tăng nhưng `activeSockets` thấp hơn nhiều so với 200: một vài tài khoản đang mở vượt quá 5 tab hoặc lỗi vòng lặp kết nối lại ở client (mã `4003`), không phải quá tải toàn cục.

Bảng 6.5. Ma trận tổng hợp kết quả kiểm chứng toàn hệ thống Grammar Quest theo phân hệ và câu hỏi nghiên cứu.

Phân hệ kiểm chứng	Tập lớp JUnit 5 tiêu biểu	Kịch bản trình duyệt (.mjs)	Số @Test	RQ / KQ
1. Điều phối WebSocket & Nhịp lập lịch	SystemSocketHandlerTest (27), SocketArena/Class/Workspace (6), RealtimeNotifier & Config (2)	check-realtime-refresh, check-account-realtime, check-notifications, check-support-* (5)	35	RQ1, RQ2 (100% Pass)
2. Lộ trình học 5 cấp, Biên tập & Nhiệm vụ	StudyRoadmapIT (1), StudyManagementValidation (4), LearningService & Roadmap (13), DailyQuest/Boss (13), Placement (5)	check-study-roadmap, check-study-management, check-daily-quests, check-placement, smoke-completion (7)	36	RQ2 (100% Pass)
3. Lớp học, Live Exam & Ngân hàng câu hỏi	LiveExamConcurrencyTest (1), LiveExamTextQuestions (5), AssignmentAnswerGuard (3), TeacherBatch/Material (12), Classroom (7)	check-live-exam, check-exam-flow, check-classroom-socket, check-teacher-library, visual-teacher-text (7)	28	RQ2, RQ3 (100% Pass)
4. Đấu trường PvP, ELO & Gói câu hỏi	ArenaMatchmakerTest (6), PvpArenaFeaturesTest (3), ArenaClockTest (1), ArenaSnapshotStoreTest (1)	check-arena-socket, check-arena-report, check-pvp-*, smoke-arena-timeout, visual-arena-match (6)	11	RQ1–RQ3 (100% Pass)
5. Sức chứa, Rate Limit & Tích hợp toàn trình	FullRealtimeIntegrationTest (13), ServerCapacityTest (2), AnswerRateLimiterTest (1), SocketSessionRegistry (1), AdminModeration (5)	check-admin-*, check-role-views, check-real-backend, smoke-admin-role, smoke-live (12)	22	RQ2–RQ4 (100% Pass)
Tổng cộng toàn dự án	35 lớp kiểm thử JUnit 5	37 kịch bản tự động (.mjs)	132	Đạt 100%

- Nếu `rejectedRequests` (`HTTP 503`) tăng kèm độ trễ cao: cả 32 khe HTTP đang bị giữ quá `1.500 ms`, báo hiệu tranh chấp khóa hàng InnoDB hoặc nghẽn bể kết nối HikariCP.
- Nếu xuất hiện cảnh báo `WEBSOCKET_HEARTBEAT_TIMEOUT` (quá 30 s không có `PONG`) hoặc `MEMORY` (heap $\geq 85\%$): phản ánh kết nối nửa mở từ mạng người dùng đang được `maintainConnections()` thu hồi hoặc cần tăng ngưỡng `-Xmx` của JVM.

Từ các giới hạn hiện tại của kiến trúc một tiến trình, ba hướng cải tiến kỹ thuật được xếp theo thứ tự ưu tiên gồm:

1. Ưu tiên 1 – Ngoại hóa trạng thái phiên và hàng đợi ghép trận sang Redis khi mở rộng ngang (Horizontal Scaling): Chuyển kênh phát tín hiệu `AFTER_COMMIT` sang Redis Pub/Sub và đưa hàng đợi `ArenaMatchmaker` cùng bộ đếm `ServerCapacity` vào Redis để nhiều bản sao Spring Boot có thể chia sẻ chung trạng thái thời gian thực.

2. Ưu tiên 2 – Truyền bản vá chênh lệch mức hàng (Row-Level Delta Patch) cho bảng bài nộp phòng thi: Thay vì gửi lại toàn bộ mảng `EXAM_SUBMISSIONS_DATA` cho giáo viên khi một học viên nộp bài, chỉ truyền bản ghi vừa thay đổi tương tự cơ chế `eve ntsDelta` của `ServerMonitorService`.

3. Ưu tiên 3 – Tách riêng luồng cập nhật đồng hồ `arenaSocket.advance()` khỏi luồng quét đẩy tin: Đưa bước chuyển vòng và tính điểm ELO của các phòng đấu PvP sang hàng đợi worker phân mảnh theo roomId mod N để giữ độ trễ của nhịp đẩy `1.000 ms` cố định khi số phòng đấu tăng cao.

Chương 7

ĐÓNG GÓP VÀ KẾT LUẬN

7.1 Đóng góp và bằng chứng

Đóng góp thứ nhất là tích hợp học tập theo tiến trình với quản lý lớp và thi đấu trong một hệ thống phân vai thống nhất. Kiểm tra đầu vào xác định điểm bắt đầu; điều kiện mở module và cấp độ. Lớp học gắn thao tác với chủ lớp hoặc thành viên, còn PvP có điều kiện tham gia riêng. Cách tổ chức này giữ liên kết qua tài khoản nhưng không đồng nhất tiến độ học với XP hoặc ELO.

Đóng góp thứ hai là tổ chức kênh giao tiếp thời gian thực có quản lý vòng đời. `SystemSocketProvider` dùng chung một WebSocket mỗi tab, liên hệ phản hồi bằng `requestId` và giới hạn yêu cầu chờ. Backend liên kết socket với phiên HTTP, xác định danh tính rồi chuyển thao tác tới dịch vụ nghiệp vụ. Các cập nhật lớp, bài kiểm tra, phòng và giám sát kết hợp sự kiện với tác vụ định kỳ.

Đóng góp thứ ba là bảo toàn ngữ cảnh đề và kết quả. Snapshot giữ nội dung gắn với lượt hoặc bài giao; `version` nhận diện đề kiểm tra hiện hành; giao dịch và khóa nghiệp vụ hỗ trợ xử lý cạnh tranh và thao tác lặp. Phân hệ bài tập lớp phân biệt điểm chính thức với lượt đầu, lượt gần nhất và lịch sử. Nhờ đó, bắt đầu làm lại không được hiểu thành xóa kết quả hợp lệ trước đó.

Đóng góp thứ tư là điều phối PvP và quản lý tài nguyên có giới hạn. Hàng đợi xét điều kiện tài khoản, cấp tài khoản và ELO, đồng thời quản lý socket sở hữu mục chờ. Gói đề được chia sẻ theo phòng và giải phóng khi hết tham chiếu. Quota kết nối, giới hạn yêu cầu và rate limit hỗ trợ từ chối có kiểm soát. Dashboard cung cấp số socket, tài khoản, hàng đợi và heap JVM để hỗ trợ chẩn đoán.

Bảng 7.1. Cơ chế và căn cứ đối chiếu chính.

Nhóm kết quả	Căn cứ trong dự án
Tiến trình học và quyền truy cập	`PlacementService`, `RoadmapCourseService` và các kiểm tra quyền lớp học
Giao tiếp và hiệu lực phiên	`SystemSocketProvider`, `SystemSocketHandler`, `SocketSessionRegistry`
Đề và kết quả nhất quán	Snapshot đề, `LiveExamService`, `AssignmentResults`, sổ XP và khóa nghiệp vụ
PvP và tài nguyên	`ArenaMatchmaker`, `ArenaSnapshotStore`, `ServerCapacity`, `AnswerRateLimiter`, `ServerMonitorService`

7.2 Trả lời câu hỏi nghiên cứu và ý nghĩa kết quả

RQ1 — Hiệu quả đồng bộ và giảm tải dư thừa

Các kết quả trình bày tại Bảng 6.2 cho thấy cơ chế gom nhóm cập nhật, bộ đệm SnapshotKey và kho gói câu hỏi ArenaSnapshotStore giúp giảm những thao tác xử lý và truyền dữ liệu lặp lại. Trong kịch bản năm tab của cùng một tài khoản theo dõi một phòng thi, bộ đệm theo khóa cho phép dùng chung kết quả truy vấn và mã hóa trong một nhịp cập nhật. Số lần mã hóa JSON giảm từ năm xuống một, tương ứng giảm 80% số lần thực hiện. Khi trạng thái không thay đổi trong 60 chu kỳ quan sát, cơ chế so sánh snapshot chỉ gửi một thông điệp ban đầu, thay vì 60 thông điệp như mô hình cơ sở, tương ứng giảm khoảng 98,3%.

Cơ chế gom nhóm DATA_INVALIDATED hợp nhất nhiều thay đổi của cùng tài khoản trong một cửa sổ xử lý, kết hợp debounce và kiểm soát yêu cầu đang thực hiện tại trình duyệt để hạn chế tải lại liên tiếp. Đối với PvP, ArenaSnapshotStore cho phép các kết nối trong cùng phòng dùng chung gói câu hỏi, tránh nạp và lưu một bản riêng cho từng kết nối. Việc nạp lại vẫn có thể xảy ra nếu gói đã được giải phóng rồi phòng được truy cập lại.

Như vậy, các cơ chế trên giảm số lần xử lý và thông điệp dư thừa trong những kịch bản đã kiểm chứng. Tuy nhiên, tỷ lệ giảm số lần mã hóa hoặc số thông điệp không đồng nghĩa với tỷ lệ giảm tương ứng về CPU, RAM, băng thông hay độ trễ toàn hệ thống. Những đại lượng này cần được đo riêng dưới tải đại diện.

RQ2 — Tính đúng đắn giao dịch và bảo toàn bất biến nghiệp vụ

Hệ thống kết hợp nhiều cơ chế để bảo vệ kết quả nghiệp vụ trước yêu cầu không hợp lệ, yêu cầu lặp và dữ liệu cũ. Khóa bi quan phối hợp các thao tác cạnh tranh trên cùng tài nguyên trong phạm vi giao dịch. Kiểm tra version và base_version phát hiện dữ liệu đã lỗi thời, hạn chế việc bản nháp hoặc yêu cầu cũ ghi đè nội dung mới. Ràng buộc duy nhất trên khóa yêu cầu và khóa phần thưởng hỗ trợ ngăn tạo lượt làm hoặc cộng thưởng lặp trong những luồng đã áp dụng.

Cơ chế AFTER_COMMIT chỉ chuyển thông báo cập nhật sang tầng thời gian thực sau khi giao dịch hoàn tất thành công, tránh công bố thay đổi của giao dịch bị rollback. Tuy nhiên, cơ chế này không tự bảo đảm thông báo luôn đến được trình duyệt; khi kết nối gián đoạn hoặc tiến trình gặp sự cố, client vẫn cần đọc lại trạng thái chính thức.

Các kết quả kiểm chứng tại mục 6.3 cho thấy những bất biến được kiểm tra đã được bảo toàn trong phạm vi kịch bản thực hiện, bao gồm kiểm soát truy cập, giữ nguyên ngữ cảnh đề và bảo vệ kết quả đã ghi nhận. Kết quả này hỗ trợ tính đúng đắn của thiết kế, nhưng chưa đủ để khẳng định dữ liệu được bảo toàn tuyệt đối trong mọi tình huống lỗi hoặc mọi thứ tự thực thi.

RQ3 — Xử lý tương tranh và phân xử xung đột trực tiếp

Trong kịch bản tại Bảng 6.4, 20 luồng học viên thực hiện lưu hoặc nộp bài đồng thời với thao tác sửa đề của giáo viên. Các thao tác liên quan sử dụng khóa trên bản ghi kỳ

kiểm tra để phối hợp cập nhật. Khi đề thay đổi, hệ thống tăng revision và thực hiện chấm lại theo quy tắc tương ứng, giúp kết quả đã lưu nhất quán với phiên bản đề được áp dụng.

Với dữ liệu thử nghiệm đã mô tả, cả 20 bài nộp đều chuyển sang trạng thái SUBMITTED và có kết quả một câu đúng trên hai câu, tương ứng 50%; không ghi nhận mất cập nhật trong kịch bản này. Việc các bài có cùng điểm là hệ quả của dữ liệu đầu vào được thiết kế cho thử nghiệm. Yêu cầu tổng quát là mỗi bài được chấm đúng theo câu trả lời của học viên và cùng quy tắc chấm, không phải mọi học viên đều nhận điểm giống nhau.

Kết quả cho thấy cơ chế giao dịch, khóa và chấm lại xử lý được tình huống tương tranh đã kiểm chứng. Tuy nhiên, thử nghiệm với 20 luồng chưa xác định thông lượng tối đa hoặc chứng minh hệ thống không thể xảy ra deadlock. Cần tiếp tục kiểm tra nhiều thứ tự thực thi, mức tải và thời gian giữ khóa, đồng thời duy trì cơ chế xử lý khi giao dịch thất bại.

RQ4 — Kiểm soát sức chứa và cô lập hành vi lạm dụng

Các thử nghiệm tại Bảng 6.4 cho thấy ServerCapacity và AnswerRateLimiter thực hiện được việc từ chối yêu cầu vượt giới hạn trong những điều kiện đã kiểm tra. Với kịch bản mở đồng thời 40 kết nối cho cùng một tài khoản, số socket được duy trì không vượt quá giới hạn năm kết nối; các kết nối vượt ngưỡng bị đóng với mã tương ứng. Khi toàn bộ 32 suất xử lý HTTP được giữ, yêu cầu kiểm tra nhận HTTP 503; sau khi giải phóng tài nguyên, yêu cầu hợp lệ được tiếp nhận trở lại.

Đối với giới hạn tốc độ, trong kịch bản một tài khoản gửi 11 câu trả lời trong khoảng thời gian dưới một giây, yêu cầu thứ 11 bị chặn theo cấu hình giới hạn 10 lượt/giây và thời gian khóa tạm thời 10 giây. Tài khoản khác dùng chung địa chỉ IP vẫn gửi được câu trả lời, cho thấy việc giới hạn theo tài khoản giúp cô lập hành vi vượt ngưỡng trong tình huống thử nghiệm.

Những kết quả này xác nhận khả năng kiểm soát hạn mức và thu hồi tài nguyên trong phạm vi kiểm thử trên một máy chủ. Chúng chưa chứng minh hệ thống có thể phục vụ ổn định 200 người dùng hoạt động đồng thời hoặc phục hồi sau mọi dạng quá tải. Việc xác định sức chứa thực tế cần kết hợp số tài khoản đang thao tác, tần suất yêu cầu, độ trễ, tỷ lệ lỗi và mức sử dụng tài nguyên dưới tải đại diện.

7.3 Giới hạn của nghiên cứu

Về kiến trúc, backend hiện là miền điều phối trung tâm. Registry socket, hàng đợi và đăng ký theo dõi nằm trong RAM của tiến trình. MySQL giữ dữ liệu đã commit nhưng không tự khôi phục toàn bộ trạng thái tạm sau restart. Hệ thống chưa có mô hình nhiều backend với kho phiên, định tuyến phòng và quyền sở hữu trạng thái được chia sẻ đầy đủ.

Về vận hành, đồng bộ kết hợp sự kiện và nhịp định kỳ nên độ mới dữ liệu phụ thuộc tải xử lý. Hiện tại còn kênh STOMP tương thích và nhiều luồng dữ liệu song song, làm tăng hợp đồng phải duy trì. WebSocket trực tiếp cấu hình `allowedOrigins("*")`;

trước khi triển khai Internet cần thu hẹp origin và kiểm tra cùng phiên, cookie, proxy, HTTPS/WSS. Dashboard tức thời chưa thay thế hệ thống giám sát dài hạn.

Về học tập, 5 cấp độ là cấu trúc nội dung nội bộ, không tự tương ứng chứng nhận CEFR. XP, ELO và số câu đúng đo các khía cạnh khác nhau của hoạt động. Chưa có nghiên cứu người dùng dài hạn hoặc nhóm đối chứng để kết luận tác động tới năng lực ngôn ngữ, động lực và thói quen học; ngân hàng câu hỏi và phần giải thích vẫn cần thẩm định chuyên môn.

7.4 Hướng phát triển

Ưu tiên đầu tiên là hoàn thiện kiểm thử toàn tuyến và cạnh tranh theo các bất biến ở chương 3. Cần thử nộp đồng thời, hai tab cùng tài khoản, nhận thưởng lặp, sửa đề khi đang làm và mất kết nối sau commit. Kiểm thử giao dịch phải dùng schema MySQL cô lập, dữ liệu xác định và kiểm tra bản ghi thực tế, tránh chỉ dựa vào phản hồi hoặc mock.

Tiếp theo là chuẩn hóa hợp đồng WebSocket: action, payload, mã lỗi, phiên bản và chính sách thử lại của thao tác ghi. Luồng khôi phục nên thống nhất theo trình tự xác nhận phiên, mở lại tài nguyên, nhận snapshot và kiểm tra trạng thái kết thúc. Đồng thời cần rà soát origin, HTTPS/WSS, cookie, log và sao lưu để hoàn thiện triển khai.

Về hiệu năng, xây dựng workload gồm tự học, kiểm tra lớp và PvP; tách số tài khoản, socket và người đang tương tác. Các phép đo cần có tỷ lệ thành công, độ trễ p50/p95/p99, thời gian truy vấn, outbound và heap sau thu hồi. Chỉ tối ưu hoặc tăng hạn mức khi có baseline, nhiều lần lặp và cùng điều kiện so sánh. Frontend có thể tách mã theo tính năng rồi đo trên thiết bị và đường truyền mục tiêu.

Về mở rộng, trước khi thêm backend cần xác định đơn vị sở hữu phòng, kho phiên, phân phối sự kiện và cách xử lý instance dừng; thêm bộ cân bằng tải đơn thuần chưa đủ. Về sản phẩm, ưu tiên thẩm định nội dung, đánh giá nhiệm vụ thực tế của học viên/giáo viên và gợi ý ôn theo lịch sử lỗi. Những mở rộng này cần đi cùng kiểm tra tính đúng và đánh giá giá trị sử dụng.

7.5 Kết luận

Đề tài Grammar Quest đã xây dựng một hệ thống học ngữ pháp tiếng Anh trực tuyến, kết hợp lộ trình tự học, luyện tập, quản lý lớp học và thi đấu đối kháng. Hệ thống đáp ứng nhu cầu của học viên, giáo viên và quản trị viên trong cùng một nền tảng, tạo sự liên kết giữa nội dung học tập, hoạt động giảng dạy và quá trình theo dõi kết quả.

Về mặt kỹ thuật, đề tài vận dụng các kiến thức về lập trình mạng và phát triển ứng dụng web để tổ chức giao tiếp client–server, cập nhật dữ liệu hai chiều và quản lý nhiều người dùng. Các cơ chế xác thực, phân quyền, lưu trữ đề và kiểm soát trạng thái giúp bảo vệ tính nhất quán của quá trình làm bài và kết quả học tập. Qua đó, đề tài thể hiện sự phối hợp giữa giao diện, xử lý nghiệp vụ, giao tiếp mạng và cơ sở dữ liệu trong việc giải quyết một bài toán thực tế.

Kết quả triển khai và kiểm thử bước đầu cho thấy tính khả thi của hướng tiếp cận. Hệ thống vẫn cần tiếp tục được đánh giá về khả năng chịu tải, phục hồi khi gián đoạn kết nối và hiệu quả sử dụng trong môi trường học tập thực tế. Việc hoàn thiện các yếu tố này, cùng với nâng cao chất lượng học liệu và trải nghiệm người dùng, sẽ tạo cơ sở để Grammar Quest phát triển thành một công cụ hỗ trợ học tập ổn định, thuận tiện và phù hợp hơn với nhu cầu giảng dạy, học tập trực tuyến.

TÀI LIỆU THAM KHẢO

[1] S. Deterding, D. Dixon, R. Khaled, and L. Nacke, "From game design elements to gamefulness: Defining gamification," in *Proceedings of the 15th International Academic MindTrek Conference: Envisioning Future Media Environments*, 2011, pp. 9–15.

[2] J. Hamari, J. Koivisto, and H. Sarsa, "Does gamification work? – A literature review of empirical studies on gamification," in *47th Hawaii International Conference on System Sciences (HICSS)*, IEEE, 2014, pp. 3025–3034.

[3] Y. W. Bernier, "Latency compensating methods in client/server in-game protocol design and optimization," in *Game Developers Conference (GDC)*, vol. 9, 2001.

[4] G. Fentham, "Networked multiplayer game architecture and latency compensation," *International Journal of Computer Games Technology*, vol. 2014, 2014.

[5] Duolingo, Inc., "The Duolingo Educational Method and Language Learning Efficacy," [Trực tuyến]. Địa chỉ: `https://www.duolingo.com/efficacy`, 2024.

[6] Kahoot! ASA, "Game-based Learning and Student Engagement Platform Guide," [Trực tuyến]. Địa chỉ: `https://kahoot.com/schools/how-it-works/`, 2024.

[7] Quizlet, Inc., "Adaptive Learning and Memory Retention Tools," [Trực tuyến]. Địa chỉ: `https://quizlet.com/features`, 2024.

[8] British Council, "LearnEnglish: English Grammar Reference and Practice Exercises," [Trực tuyến]. Địa chỉ: `https://learnenglish.britishcouncil.org/grammar`, 2024.

[9] I. Fette and A. Melnikov, "The WebSocket Protocol," RFC 6455, Internet Engineering Task Force (IETF), 2011. [Trực tuyến]. Địa chỉ: `https://datatracker.ietf.org/doc/html/rfc6455`.

[10] VMware Tanzu, "Spring Boot Reference Documentation (Version 3.5)," VMware, Inc., [Trực tuyến]. Địa chỉ: `https://docs.spring.io/spring-boot/docs/current/reference/html/`, 2024.

[11] Meta Platforms, Inc., "React: The Library for Web and Native User Interfaces," [Trực tuyến]. Địa chỉ: `https://react.dev/`, 2024.

[12] Oracle Corporation, "MySQL 8.0 Reference Manual," Oracle, [Trực tuyến]. Địa chỉ: https://dev.mysql.com/doc/refman/8.0/en/, 2024.

[13] Redgate Software, "Flyway Database Migrations Documentation," [Trực tuyến]. Địa chỉ: https://documentation.red-gate.com/fd, 2024.